

Systemy Baz Danych 2023/2024

Projekt

Autorzy:

- Bartłomiej Treśka
- Robert Mesek
- Jakub Mrozowski

Użytkownicy.....	6
Wymagania	7
Schemat	9
Tabele.....	10
'users'	10
'adress_details'	11
'countries_citieses'	12
'user_types'	13
'customers'	14
'orders'	15
'order_details'	16
'pending_payments'	17
'completed_payments'	18
'service_types'	19
'teachers'	20
'translators'	21
'languages'	22
'studies'	23
'studies_class_translators'	24
'studies_classes'	25
'studies_classroom_classes'	26
'studies_online_classes'	27
'studies_passes'	28
'studies_practice_modules'	29
'studies_presences'	30
'studies_price_per_classes'	31
'classroom_details'	32
'courses'	33
'course_classroom_modules'	34
'course_module_types'	35
'course_modules'	36
'course_online_modules'	37
'course_presences'	38

'course_translations'.....	39
'course_types'	40
'webinars'	41
'webinar_types'	43
'webinars_translations'	44
Widoki	45
'basket_products' - RM.....	45
'user_completed_payment_details' - RM	46
'events_purchase_deadline' - RM	47
'user_pending_payment_details' - RM	48
'presences_view' - RM	49
'events'	51
'studies_events' - BT	52
'presence_on_finished_events' - BT.....	53
'event_participants' - BT	54
'profit_by_service_type' - BT.....	55
'absence_list_courses' - JM	56
'absence_list_studies_classes' - JM.....	57
'bilocation_raport' - BT	58
'debtors' - JM	59
'orders_for_user' - JM.....	60
'price_courses_orders' - JM.....	61
'price_studies_orders' - JM.....	62
'price_webinars_orders' - JM	63
'price_for_customers_orders' - JM.....	64
'shipping_details' - JM	65
'table_studies' - JM.....	67
'table_webinars' - JM.....	68
'table_courses' - JM	69
'table_practicies' - JM.....	70
'time_table' - JM,BT.....	71
'staff_time_table' - BT	73
Typy danych	74
'SERVICES_TABLE_TYPE' - RM.....	74

'INSTALMENTS_TABLE_TYPE' - RM.....	74
Funkcje	75
'fn_sum_services_table' - RM	75
'fn_has_access(@customer_id INT, @service_id INT, @service_type_id INT)' - RM	76
'fn_calc_enrollment(@service_id INT, @service_type_id INT)' - RM	77
'fn_get_deadline(@service_id INT, @service_type_id INT)' - RM	78
'fn_calc_limit_left(@service_id INT, @service_type_id INT)' - RM	79
fn_calc_total_order_value(@order_id) BT.....	82
fn_staff_has_free_time(@employee_id, @start_time, @end_time) BT.....	83
fn_get_price_of_service(@service_id, @service_type_id) BT.....	84
fn_get_presence(@customer_id, @service_id, @service_type_id) BT.....	85
fn_get_meeting_type(@service_id, @service_type_id) BT.....	87
fn_get_study_class_type(@class_id) - BT	88
fn_get_number_of_college_classes(@studies_id) BT	89
fn_calc_average_order_value() BT.....	90
fn_get_most_expensive_order_price() BT	91
fn_get_least_expensive_order_price() BT	92
'fn_check_completed_payment' - RM	93
'fn_check_pending_payment' - RM.....	94
'fn_sum_services_table' - RM	95
'fn_get_bilocation_for_user' - BT.....	96
'fn_get_time_table_for_user' - BT	97
'fn_get_debt_for_user' - BT	98
'fn_get_staff_time_table_for_translator' - BT	100
Procedury.....	102
'usp_orders_insert' - RM	102
'usp_delete_pending_order' - RM.....	105
'usp_complete_payment' - RM	107
usp_user_insert' BT	108
'usp_teacher_insert' BT.....	110
'usp_translator_insert' BT	111
'usp_webinar_insert' BT	112
'usp_course_insert' BT.....	114
'usp_course_module_insert' BT	116

‘usp_studies_insert’ BT	118
‘usp_studies_class_insert’ BT	120
‘usp_change_address’ BT	122
‘usp_change_user_data’ BT	123
‘usp_add_presence’ BT	124
Wyzwalacze	125
‘trg_delete_details_on_order_delete’ - RM	125
Role	126
‘Accountant’- BT	126
‘principal’- BT	127
‘customer’- BT	128
‘Teacher’- BT	129
‘Translator’- BT	130
‘unsigned_customer’- BT	131
Indeksy	132
‘Indeks service_types’ - JM	132
‘Indeks studies_limitation’ - JM	132
‘Indeks teacher_for_studies’- JM	132
‘Indeks teacher_for_webinars’- JM	132
‘Indeks teacher_language’- JM	132
‘Indeks studies_practice’- JM	132
‘Indeks user_adress’- JM	132
‘Indeks user_login’- JM	132
‘Indeks user_contact’- JM	133
‘Indeks com_inf’- JM	133
‘Indeks course_list’- JM	133
‘Indeks webinar_list’- JM	133
‘Indeks studie_list’- JM	133
‘Indeks practice_list’- JM	133
‘Indeks course_module_list’- JM	133
‘Indeks pending_pay’- JM	133
Przykłady	135
‘Przykładowe działanie koszyka.’ - RM	135

Użytkownicy

- Administratorzy:
 - Administrator bazy danych (kilku)
 - Dyrektor Szkoły (zastępcy mają tę samą rolę)
- Pracownik:
 - Księgowy (kilku)
 - Nauczyciel
 - Tłumacz
- Klient (Uczeń):
 - Uczestnik kursu
 - Uczestnik webinarium
 - Uczestnik studiów
 - Zarejestrowany użytkownik
- Niezalogowany użytkownik

Wymagania

Każdy użytkownik dziedziczy funkcje w wyniku następującej hierarchii:

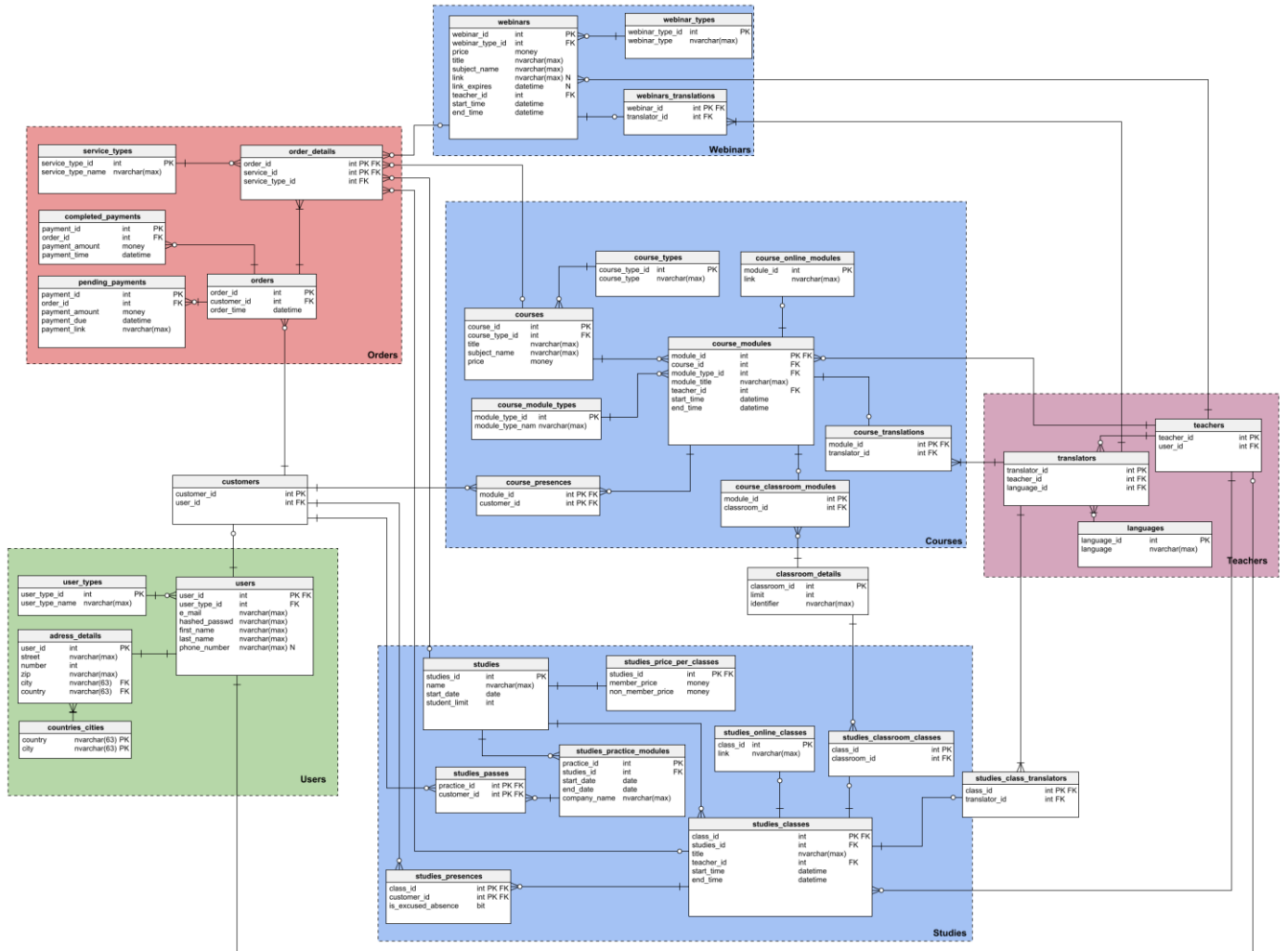
Admin > Zarząd > Pracownik > Klient

Funkcje użytkowników:

- ❖ Niezalogowany użytkownik:
 - Założenie konta
 - Logowanie na konto
 - Przeglądanie oferty
- ❖ Klient:
 - Zarządzanie kontem:
 - Założenie konta
 - Logowanie, wylogowanie z konta
 - Usunięcie konta
 - Modyfikowanie konta (np. danych)
 - Podgląd historii zakupów
 - Podgląd aktualnych subskrypcji
 - Zarządzanie aktualnymi subskrypcjami
 - Wygenerowanie grafiku
 - Zakup dostępu do zajęć
 - Możliwość oceny jakości usługi
- ❖ Uczestnik studium:
 - Dostęp do praktyk
- ❖ Pracownik:
 - Ocenianie studentów
 - Sprawdzanie obecności
- ❖ Księgowy:
 - Generowanie raportów
 - Raport finansowy
 - Lista dłużników
 - Statystyki zapisów dla wydarzenia
 - Frekwencja na zakończonych wydarzeniach
 - Lista obecności dla danych szkoleń
 - Raport bilokacji
 - Generowanie listy stałych klientów
 - Wygenerowanie dyplomu
- ❖ Admin:
 - Zarządzanie kursami, webinariami, studiami:
 - Dodanie
 - Usunięcie
 - Modyfikacja
- ❖ Ponadto administratorzy posiadają dostęp do wszystkich funkcji.
- ❖ Funkcje systemowe:

- ❖ Integracja z systemem płatności
 - Generowanie danych do płatności (kwota, rabat, termin do zapłaty, link do płatności)
 - Nadanie dostępu uczniowi do kursu
 - Rejestracja pomyślnych transakcji
- ❖ Dodatkowe funkcje
 - Zapisywanie nadchodzących płatności
 - Generowanie ceny za pojedyncze zajęcia spoza grona stałych klientów studium
 - Generowanie statystyk zadowolenia usług
 - Cotygodniowy backup bazy danych

Schemat



Tabele

'users'

Przechowuje informację o użytkownikach bazy danych.

Column name	Type	Properties	Description
user_id	int	PK	id użytkownika
user_type_id	int		id typu użytkownika
e_mail	nvarchar(max)		e-mail użytkownika
hashed_passwd	nvarchar(max)		hasło do konta użytkownika
first_name	nvarchar(max)		imię użytkownika
last_name	nvarchar(max)		nazwisko użytkownika
phone_number	nvarchar(max)	null	numer telefonu użytkownika
Column		Check	
e_mail nvarchar(max)		e_mail like '%@%.%'	
phone_number nvarchar(max)		ISNUMERIC(phone_number)	

```
CREATE TABLE users (  
    user_id int NOT NULL,  
    user_type_id int NOT NULL,  
    e_mail nvarchar(max) NOT NULL CHECK (e_mail like '%@%.%'),  
    hashed_passwd nvarchar(max) NOT NULL,  
    first_name nvarchar(max) NOT NULL,  
    last_name nvarchar(max) NOT NULL,  
    phone_number nvarchar(max) NULL DEFAULT ISNUMERIC(phone_number),  
    CONSTRAINT unique_email UNIQUE (e_mail),  
    CONSTRAINT users_pk PRIMARY KEY (user_id)  
);
```

'adress_details'

Adresy użytkowników.

Column name	Type	Properties	Description
user_id	int	PK	id użytkownika
street	nvarchar(max)		ulica
number	int		numer domu
zip	nvarchar(max)		numer zip
city	nvarchar(max)		miasto
country	nvarchar(max)		państwo
Column		Check	
number int		number >= 1	

```
CREATE TABLE adress_details (  
    user_id int NOT NULL,  
    street nvarchar(max) NOT NULL,  
    number int NOT NULL CHECK (number >= 1),  
    zip nvarchar(max) NOT NULL,  
    city nvarchar(63) NOT NULL,  
    country nvarchar(63) NOT NULL,  
    CONSTRAINT adress_details_pk PRIMARY KEY (user_id)  
);
```

‘countries_citieses’

Słownik z miastami w państwach

Column name	Type	Properties	Description
country	nvarchar(63)	PK	Nazwa państwa
city	nvarchar(63)	PK	Nazwa miasta

```
CREATE TABLE countries_cities (  
    country nvarchar(63) NOT NULL,  
    city nvarchar(63) NOT NULL,  
    CONSTRAINT countries_cities_pk PRIMARY KEY (country,city)  
);
```

'user_types'

Typy użytkowników.

Column name	Type	Properties	Description
user_type_id	int	PK	id typu użytkownika
user_type_name	nvarchar(max)		typ użytkownika

```
CREATE TABLE user_types (  
    user_type_id int NOT NULL,  
    user_type_name nvarchar(max) NOT NULL,  
    CONSTRAINT user_types_pk PRIMARY KEY (user_type_id)  
);
```

'customers'

Przechowuje klientów.

Column name	Type	Properties	Description
customer_id	int	PK	id klienta
user_id	int		id użytkownika

```
CREATE TABLE customers (  
    customer_id int NOT NULL,  
    user_id int NOT NULL,  
    CONSTRAINT customers_pk PRIMARY KEY (customer_id)  
);
```

'orders'

Dokonane zamówienia.

Column name	Type	Properties	Description
order_id	int	PK	id zamówienia
customer_id	int		id klienta
order_time	datetime	default GETDATE()	godzina i data zamówienia
Column		Check	
Order_time datetime		Year(order_time) >= 2020	

```
CREATE TABLE orders (  
  order_id int NOT NULL IDENTITY(1, 1),  
  customer_id int NOT NULL,  
  order_time datetime NOT NULL DEFAULT GETDATE() CHECK (year(order_time) >= 2020),  
  CONSTRAINT orders_pk PRIMARY KEY (order_id)  
);
```

'order_details'

Szczegóły zamówień.

Column name	Type	Properties	Description
order_id	int	PK	id zamówienia
service_id	int	PK	id zakupionego usługi
service_type_id	int		id typu zakupionego usługi

```
CREATE TABLE order_details (  
  order_id int NOT NULL,  
  service_id int NOT NULL,  
  service_type_id int NOT NULL,  
  CONSTRAINT order_details_pk PRIMARY KEY (order_id,service_id)  
);
```

'pending_payments'

Płatności czekające na dokończenie.

Column name	Type	Properties	Description
payment_id	int	PK	id płatności
order_id	int		id zamówienia
payment_amount	money		kwota płatności
payment_due	datetime		do kiedy należy zapłacić
payment_link	nvarchar(max)		link do płatności
Column		Check	
payment_amount money		payment_amount >= 0	
payment_due datetime		Year(payment_due) >= 2020	

```
CREATE TABLE pending_payments (  
    payment_id int NOT NULL,  
    order_id int NOT NULL,  
    payment_amount money NOT NULL CHECK (payment_amount >= 0),  
    payment_due datetime NOT NULL CHECK (year(payment_due) >= 2020),  
    payment_link nvarchar(max) NOT NULL,  
    CONSTRAINT pending_payments_pk PRIMARY KEY (payment_id)  
);
```

'completed_payments'

Ukończone płatności.

Column name	Type	Properties	Description
payment_id	int	PK	id płatności
order_id	int		id zamówienia
payment_amount	money		kwota płatności
payment_time	datetime		data płatności
Column		Check	
payment_amount money		payment_amount >= 0	
payment_time datetime		Year(payment_time) >= 2020	

```
CREATE TABLE completed_payments (  
  payment_id int NOT NULL,  
  order_id int NOT NULL,  
  payment_amount money NOT NULL CHECK (payment_amount >= 0),  
  payment_time datetime NOT NULL CHECK (year(payment_time) >= 2020),  
  CONSTRAINT completed_payments_pk PRIMARY KEY (payment_id)  
);
```

'service_types'

Rodzaje usług.

Column name	Type	Properties	Description
service_type_id	int	PK	id usługi
service_type_name	nvarchar(max)		nazwa usługi

```
CREATE TABLE service_types (  
    service_type_id int NOT NULL,  
    service_type_name nvarchar(max) NOT NULL,  
    CONSTRAINT service_types_pk PRIMARY KEY (service_type_id)  
);
```

'teachers'

Użytkownicy, którzy są nauczycielami.

Column name	Type	Properties	Description
teacher_id	int	PK	id nauczyciela
user_id	int		id użytkownika

```
CREATE TABLE teachers (  
  teacher_id int NOT NULL,  
  user_id int NOT NULL,  
  CONSTRAINT teachers_pk PRIMARY KEY (teacher_id)  
);
```

'translators'

Tłumacze z podziałem na języki.

Column name	Type	Properties	Description
translator_id	int	PK	id tłumacza
teacher_id	int		id nauczyciela
language_id	int		id języka tłumaczonego

```
CREATE TABLE translators (  
  translator_id int NOT NULL,  
  teacher_id int NOT NULL,  
  language_id int NOT NULL,  
  CONSTRAINT translators_pk PRIMARY KEY (translator_id)  
);
```

'languages'

Słownik języków.

Column name	Type	Properties	Description
language_id	int	PK	id języka
language	nvarchar(max)		nazwa języka

```
CREATE TABLE languages (  
    language_id int NOT NULL,  
    language nvarchar(max) NOT NULL,  
    CONSTRAINT languages_pk PRIMARY KEY (language_id)  
);
```

'studies'

Wszystkie studia.

Column name	Type	Properties	Description
studies_id	int	PK	id studiów
name	nvarchar(max)		nazwa studiów
start_date	date		data startowa
student_limit	int		limit studentów
Column		Check	
start_date date		year(start_date) >= 2020	
student_limit int		student_limit >= 0	

```
CREATE TABLE studies (  
    studies_id int NOT NULL,  
    name nvarchar(max) NOT NULL,  
    start_date date NOT NULL CHECK (year(start_date) >= 2020),  
    student_limit int NOT NULL CHECK (student_limit >= 0),  
    CONSTRAINT studies_pk PRIMARY KEY (studies_id)  
);
```

'studies_class_translators'

Tłumaczenia zajęć na studiach.

Column name	Type	Properties	Description
class_id	int	PK	id zajęć
translator_id	int		id tłumacza

```
CREATE TABLE studies_class_translators (  
    class_id int NOT NULL,  
    translator_id int NOT NULL,  
    CONSTRAINT studies_class_translators_pk PRIMARY KEY (class_id)  
);
```

'studies_classes'

Zajęcia na studiach.

Column name	Type	Properties	Description
class_id	int	PK	id zajęć
studies_id	int		id studiów
title	nvarchar(max)		nazwa zajęć
teacher_id	int		id nauczyciela
start_time	datetime		godzina i data startu
end_time	datetime		godzina i data zakończenia
Column		Check	
start_time datetime		Year(start_time) >= 2020	
end_time datetime		Year(end_time) >= 2020 end_time > start_time	

```
CREATE TABLE studies_classes (  
    class_id int NOT NULL,  
    studies_id int NOT NULL,  
    title nvarchar(max) NOT NULL,  
    teacher_id int NOT NULL,  
    start_time datetime NOT NULL CHECK (year(start_time) >= 2020),  
    end_time datetime NOT NULL CHECK (year(end_time) >= 2020),  
    CONSTRAINT end_time_po_start_time CHECK (end_time > start_time),  
    CONSTRAINT studies_classes_pk PRIMARY KEY (class_id)  
);
```

'studies_classroom_classes'

Zajęcia stacjonarne na studiach.

Column name	Type	Properties	Description
class_id	int	PK	id zajęć
classroom_id	int		id sali lekcyjnej

```
CREATE TABLE studies_classroom_classes (  
    class_id int NOT NULL,  
    classroom_id int NOT NULL,  
    CONSTRAINT studies_classroom_classes_pk PRIMARY KEY (class_id)  
);
```

'studies_online_classes'

Zajęcia online na studiach.

Column name	Type	Properties	Description
class_id	int	PK	id zajęć
link	nvarchar(max)		link do zajęć

```
CREATE TABLE studies_online_classes (  
    class_id int NOT NULL,  
    link nvarchar(max) NOT NULL,  
    CONSTRAINT studies_online_classes_pk PRIMARY KEY (class_id)  
);
```

'studies_passes'

Zaliczenia praktyk.

Column name	Type	Properties	Description
practice_id	int	PK	id praktyk
customer_id	int	PK	id klienta

```
CREATE TABLE studies_passes (  
  practice_id int NOT NULL,  
  customer_id int NOT NULL,  
  CONSTRAINT studies_passes_pk PRIMARY KEY (practice_id,customer_id)  
);
```

'studies_practice_modules'

Praktyki na studiach.

Column name	Type	Properties	Description
practice_id	int	PK	id praktyk
studies_id	int		id studiów
start_date	date		data startu
end_date	date		data zakończenia
company_name	nvarchar(max)		nazwa firmy udzielającej praktyki
Column		Check	
start_date date		Year(start_date) >= 2020	
end_date date		Year(end_date) >= 2020 end_date > start_date	

```
CREATE TABLE studies_practice_modules (  
  practice_id int NOT NULL,  
  studies_id int NOT NULL,  
  start_date date NOT NULL CHECK (year(start_date) >= 2020),  
  end_date date NOT NULL CHECK (year(end_date) >= 2020),  
  company_name nvarchar(max) NOT NULL,  
  CONSTRAINT end_date_po_start_date CHECK (end_date > start_date),  
  CONSTRAINT studies_practice_modules_pk PRIMARY KEY (practice_id)  
);
```

'studies_presences'

Obecność na zajęciach na studiach.

Column name	Type	Properties	Description
class_id	int	PK	id zajęć
customer_id	int	PK	id klienta
is_excused_absence	bit	default 0	czy to usprawiedliwienie

```
CREATE TABLE studies_presences (  
  class_id int NOT NULL,  
  customer_id int NOT NULL,  
  is_excused_absence bit NOT NULL DEFAULT 0,  
  CONSTRAINT studies_presences_pk PRIMARY KEY (class_id,customer_id)  
);
```

'studies_price_per_classes'

Ceny za jedno zajęcia ze studiów.

Column name	Type	Properties	Description
studies_id	int	PK	id studiów
member_price	money		cena dla studenta
non_member_price	money		cena dla nie studenta
Column		Check	
member_price money		member_price >= 0	
non_member_price money		non_member_price >= 0 non_member_price >= member_price	

```
CREATE TABLE studies_price_per_classes (  
  studies_id int NOT NULL,  
  member_price money NOT NULL CHECK (member_price >= 0),  
  non_member_price money NOT NULL CHECK (non_member_price >= 0),  
  CONSTRAINT nonMemberPrice_wieksza_od_memberPrice CHECK (non_member_price >=  
member_price),  
  CONSTRAINT studies_price_per_classes_pk PRIMARY KEY (studies_id)  
);
```

'classroom_details'

Szczegóły danej sali lekcyjnej.

Column name	Type	Properties	Description
classroom_id	int	PK	id sali lekcyjnej
limit	int		limit miejsc
identifier	nvarchar(max)		identyfikator pomieszczenia
Column	Check		
Limit int	limit >= 0		

```
CREATE TABLE classroom_details (  
    classroom_id int NOT NULL,  
    limit int NOT NULL CHECK (limit>=0),  
    identifier nvarchar(max) NOT NULL,  
    CONSTRAINT classroom_details_pk PRIMARY KEY (classroom_id)  
);
```

'courses'

Wszystkie kursy.

Column name	Type	Properties	Description
course_id	int	PK	id kursu
course_type_id	int		id typu kursu
title	nvarchar(max)		nazwa kursu
subject_name	nvarchar(max)		nazwa przedmiotu
price	money		cena kursu
Column		Check	
price money		price >= 0	

```
CREATE TABLE courses (  
    course_id int NOT NULL,  
    course_type_id int NOT NULL,  
    title nvarchar(max) NOT NULL,  
    subject_name nvarchar(max) NOT NULL,  
    price money NOT NULL CHECK (price >= 0),  
    CONSTRAINT courses_pk PRIMARY KEY (course_id)  
);
```

'course_classroom_modules'

Moduły kursów stacjonarnych.

Column name	Type	Properties	Description
module_id	int	PK	id modulu
classroom_id	int		id sali lekcyjnej

```
CREATE TABLE course_classroom_modules (  
  module_id int NOT NULL,  
  classroom_id int NOT NULL,  
  CONSTRAINT course_classroom_modules_pk PRIMARY KEY (module_id)  
);
```

'course_module_types'

Słownik z typami pojedynczych spotkań kursowych.

Column name	Type	Properties	Description
module_type_id	int	PK	Id typu modułu
module_type_name	nvarchar(max)		Nazwa typu modułu

```
CREATE TABLE course_module_types (  
    module_type_id int NOT NULL,  
    module_type_name nvarchar(max) NOT NULL,  
    CONSTRAINT course_module_types_pk PRIMARY KEY (module_type_id)  
);
```

'course_modules'

Moduły kursów.

Column name	Type	Properties	Description
module_id	int	PK	id modulu kursu
course_id	int		id kursu
module_type_id	int		typ modulu kursu
module_title	nvarchar(max)		nazwa modulu
teacher_id	int		id nauczyciela
start_time	datetime		data i godzina rozpoczęcia modulu
end_time	datetime		data i godzina zakończenia modulu

```
CREATE TABLE course_modules (  
    module_id int NOT NULL,  
    course_id int NOT NULL,  
    module_type_id int NOT NULL,  
    module_title nvarchar(max) NOT NULL,  
    teacher_id int NOT NULL,  
    start_time datetime NOT NULL CHECK (year(start_time) >= 2020),  
    end_time datetime NOT NULL CHECK (year(end_time) >= 2020),  
    CONSTRAINT end_time_po_start_time CHECK (end_time > start_time),  
    CONSTRAINT course_modules_pk PRIMARY KEY (module_id)  
);
```

'course_online_modules'

Moduły kursów online.

Column name	Type	Properties	Description
module_id	int	PK	id modulu kursu
link	nvarchar(max)		link do modulu kursu

```
CREATE TABLE course_online_modules (  
    module_id int NOT NULL,  
    link nvarchar(max) NOT NULL,  
    CONSTRAINT course_online_modules_pk PRIMARY KEY (module_id)  
);
```

'course_presences'

Obecności na kursach.

Column name	Type	Properties	Description
module_id	int	PK	id modulu
customer_id	int	PK	id klienta

```
CREATE TABLE course_presences (  
    module_id int NOT NULL,  
    customer_id int NOT NULL,  
    CONSTRAINT course_presences_pk PRIMARY KEY (module_id,customer_id)  
);
```

'course_translations'

Tłumaczenia kursów.

Column name	Type	Properties	Description
module_id	int	PK	id modulu
translator_id	int		id tłumacza modulu

```
CREATE TABLE course_translations (  
  module_id int NOT NULL,  
  translator_id int NOT NULL,  
  CONSTRAINT course_translations_pk PRIMARY KEY (module_id)  
);
```

'course_types'

Typu kursów.

Column name	Type	Properties	Description
course_type_id	int	PK	id typu kursu
course_type	nvarchar(max)		nazwa typu kursu

```
CREATE TABLE course_types (  
    course_type_id int NOT NULL,  
    course_type nvarchar(max) NOT NULL,  
    CONSTRAINT course_types_pk PRIMARY KEY (course_type_id)  
);
```

'webinars'

Wszystkie webinaria.

Column name	Type	Properties	Description
webinar_id	int	PK	id webinarium
webinar_type_id	int		id typu webinaru
price	money		cena webinaru
title	nvarchar(max)		nazwa webinarium
subject_name	nvarchar(max)		nazwa przedmiotu
link	nvarchar(max)	null	link do webinarium
link_expires	datetime	null default getdate()	data wygaśnięcia linku
teacher_id	int		id nauczyciela
start_time	datetime		data startu webinarium
end_time	datetime		koniec webinarium
Column		Check	
price money		price >= 0	
start_time datetime		Year(start_time) >= 2020	
end_time datetime		Year(end_time) >= 2020	
start_time datetime, end_time datetime		end_time > start_time	

```
CREATE TABLE webinars (  
    webinar_id int NOT NULL,  
    webinar_type_id int NOT NULL,  
    price money NOT NULL CHECK (price >= 0),  
    title nvarchar(max) NOT NULL,  
    subject_name nvarchar(max) NOT NULL,  
    link nvarchar(max) NULL,  
    link_expires datetime NULL DEFAULT GETDATE(),  
    teacher_id int NOT NULL,  
    start_time datetime NOT NULL CHECK (year(start_time) >= 2020),  
    end_time datetime NOT NULL CHECK (year(end_time) >= 2020),  
    CONSTRAINT end_time_po_start_time CHECK (end_time > start_time),
```

```
CONSTRAINT webinars_pk PRIMARY KEY (webinar_id)  
);
```

'webinar_types'

Typy webinarów.

Column name	Type	Properties	Description
webinar_type_id	int	PK	id typu webinarium
webinar_type	nvarchar(max)		nazwa typu webinarium

```
CREATE TABLE webinar_types (  
    webinar_type_id int NOT NULL,  
    webinar_type nvarchar(max) NOT NULL,  
    CONSTRAINT webinar_types_pk PRIMARY KEY (webinar_type_id)  
);
```

'webinars_translations'

Tłumaczenia webinarów.

Column name	Type	Properties	Description
webinar_id	int	PK	id webinarium
translator_id	int		id tłumacza webinarium

```
CREATE TABLE webinars_translations (  
  webinar_id int NOT NULL,  
  translator_id int NOT NULL,  
  CONSTRAINT webinars_translations_pk PRIMARY KEY (webinar_id)  
);
```

Widoki

'basket_products' - RM

Wyświetla koszyk z informacjami o dodanych produktach.

```
CREATE VIEW basket_products AS
SELECT    o.order_id,
          e.service_type,
          e.title,
          e.[online/stationary],
          e.start_time,
          e.end_time,
          pp.payment_id,
          pp.payment_amount,
          pp.payment_due,
          pp.payment_link
FROM      order_details od
JOIN      orders o ON o.order_id = od.order_id
JOIN      pending_payments pp ON pp.order_id = o.order_id
JOIN      service_types st ON st.service_type_id = od.service_type_id
JOIN      dbo.events e ON e.service_id = od.service_id
AND       e.service_type = st.service_type_name
```

'user_completed_payment_details' - RM

Wyświetla produkty z ukończonych zamówień dla każdego klienta.

```
CREATE VIEW user_completed_payment_details AS
SELECT orders.customer_id,
       completed_payments.payment_id,
       completed_payments.order_id,
       order_details.service_id,
       order_details.service_type_id,
       completed_payments.payment_time
FROM completed_payments
JOIN orders ON completed_payments.order_id = orders.order_id
JOIN order_details ON orders.order_id = order_details.order_id
```

'events_purchase_deadline' - RM

Wyświetla limity zapisów oraz ostateczną datę, do której możemy się zapisać na kurs.

```
CREATE VIEW events_purchase_deadline AS
SELECT webinar_id AS 'service_id',
       1 AS 'service_type_id',
       start_time AS 'purchase_deadline',
       NULL AS 'limit'
FROM webinars
UNION
SELECT course_id,
       2,
       DATEADD(DAY, -3, (
         SELECT MIN(start_time)
         FROM course_modules
         WHERE courses.course_id = course_modules.course_id
       )) AS 'purchase_deadline',
       (SELECT MIN(cd.limit)
        FROM course_modules cm JOIN course_classroom_modules ccm
        ON cm.module_id=ccm.module_id JOIN classroom_details cd
        ON cd.classroom_id=ccm.classroom_id
        WHERE cm.course_id=courses.course_id)
       AS 'limit'
FROM courses
UNION
SELECT studies_id,
       3,
       DATEADD(DAY, -3, (
         SELECT MIN(start_time)
         FROM studies_classes
         WHERE studies.studies_id = studies_classes.studies_id
       )) AS 'purchase_deadline',
       studies.student_limit AS 'limit'
FROM studies
UNION
SELECT class_id,
       4,
       DATEADD(DAY, -3, studies_classes.start_time) AS 'purchase_deadline',
       (SELECT MIN(cd.limit)
        FROM studies_classes sc JOIN studies_classroom_classes scc
        ON sc.class_id=scc.class_id JOIN classroom_details cd
        ON cd.classroom_id=scc.classroom_id)
       AS 'limit'
FROM studies_classes
GO
```

'user_pending_payment_details' - RM

Wyświetla produkty z zamówień oczekujących na płatność dla każdego klienta.

```
CREATE VIEW user_pending_payment_details AS
SELECT orders.customer_id,
       pending_payments.payment_id,
       pending_payments.order_id,
       order_details.service_id,
       order_details.service_type_id,
       pending_payments.payment_due
FROM pending_payments
JOIN orders ON pending_payments.order_id = orders.order_id
JOIN order_details ON orders.order_id = order_details.order_id
```

'presences_view' - RM

Widok z listą obecności dla każdego szkolenia.

```
CREATE VIEW presences_view AS
SELECT
ofu.service_id AS 'event_id',
ofu.service_type_id AS 'event_type_id',
'webinar' AS 'event_name',
w.title AS 'title',
1 AS 'is_online',
w.start_time AS 'start_time',
w.end_time AS 'end_time',
ofu.customer_id AS 'customer_id',
u.first_name AS 'first_name',
u.last_name AS 'last_name',
1 AS 'was_present'
FROM webinars w
JOIN dbo.orders_for_user ofu ON w.webinar_id = ofu.service_id AND ofu.service_type_id=1
JOIN customers cus ON cus.customer_id=ofu.customer_id
JOIN users u ON u.user_id=cus.user_id

UNION

SELECT
ofu.service_id AS 'event_id',
ofu.service_type_id AS 'event_type_id',
'course' AS 'event_name',
(c.title + ': ' + cm.module_title) AS 'title',
IIF(cm.module_type_id=2, 1, 0) AS 'is_online',
cm.start_time AS 'start_time',
cm.end_time AS 'end_time',
ofu.customer_id AS 'customer_id',
u.first_name AS 'first_name',
u.last_name AS 'last_name',
IIF(EXISTS (SELECT * FROM course_presences WHERE customer_id=ofu.customer_id), 1, 0) AS
'was_present'
FROM course_modules cm
JOIN courses c ON c.course_id=cm.course_id
JOIN dbo.orders_for_user ofu ON c.course_id = ofu.service_id AND ofu.service_type_id=2
JOIN customers cus ON cus.customer_id=ofu.customer_id
JOIN users u ON u.user_id=cus.user_id

UNION

SELECT
ofu.service_id AS 'event_id',
ofu.service_type_id AS 'event_type_id',
```

```

'studies_class' AS 'event_name',
(s.name + ': ' + sc.title) AS 'title',
IIF(EXISTS (SELECT * FROM studies_online_classes soc WHERE soc.class_id=sc.class_id), 1,
0) AS 'is_online',
sc.start_time AS 'start_time',
sc.end_time AS 'end_time',
ofu.customer_id AS 'customer_id',
u.first_name AS 'first_name',
u.last_name AS 'last_name',
IIF(EXISTS (SELECT * FROM studies_presences WHERE customer_id=ofu.customer_id), 1, 0) AS
'was_present'
FROM studies_classes sc
JOIN studies s ON s.studies_id=sc.studies_id
JOIN dbo.orders_for_user ofu ON s.studies_id = ofu.service_id AND ofu.service_type_id=3
JOIN customers cus ON cus.customer_id=ofu.customer_id
JOIN users u ON u.user_id=cus.user_id

UNION

SELECT
ofu.service_id AS 'event_id',
ofu.service_type_id AS 'event_type_id',
'practice' AS 'event_name',
('Practice at: ' + spm.company_name) AS 'title',
0 AS 'is_online',
CAST(spm.start_date AS DATETIME) AS 'start_time',
CAST(spm.end_date AS DATETIME) AS 'end_time',
ofu.customer_id AS 'customer_id',
u.first_name AS 'first_name',
u.last_name AS 'last_name',
IIF(EXISTS (SELECT * FROM studies_passes WHERE customer_id=ofu.customer_id), 1, 0) AS
'was_present'
FROM studies_practice_modules spm
JOIN studies s ON spm.studies_id=s.studies_id
JOIN dbo.orders_for_user ofu ON s.studies_id = ofu.service_id AND ofu.service_type_id=3
JOIN customers cus ON cus.customer_id=ofu.customer_id
JOIN users u ON u.user_id=cus.user_id

GO

```

'events'

Wypisuje wszystkie wydarzenia w jakich można / można było zakupić udział

```
CREATE VIEW [dbo].[events]
AS
    SELECT webinar_id as 'service_id', 'webinar' as 'service_type', 1 as
'service_type_id',
        webinar_type as 'online/stationary', price, title, start_time, end_time FROM
webinars
    INNER JOIN webinar_types ON webinars.webinar_type_id = webinar_types.webinar_type_id
    UNION
    SELECT course_id, 'course', 2, course_type, price, title, (
        SELECT MIN(start_time) from course_modules WHERE
courses.course_id=course_modules.course_id
    ) as 'start_time', (
        SELECT MAX(end_time) from course_modules WHERE
courses.course_id=course_modules.course_id
    ) FROM courses
    INNER JOIN course_types ON courses.course_type_id = course_types.course_type_id
    UNION
    SELECT studies_id, 'studies', 3, 'Hybrid', (
        SELECT dbo.fn_get_number_of_college_classes(studies_id)*member_price as
'class_number'
        FROM studies_price_per_classes WHERE
studies.studies_id=studies_price_per_classes.studies_id
    ) as 'price', name, start_date, (
        SELECT MAX(end_time) FROM studies_classes WHERE
studies.studies_id=studies_classes.studies_id
    ) as 'end_time'
    FROM studies
    UNION
    SELECT class_id, 'studies_classes', 4, dbo.fn_get_study_class_type(class_id),
(SELECT member_price FROM studies_price_per_classes WHERE
studies_classes.studies_id=studies_price_per_classes.studies_id) as 'price',
studies_classes.title, studies_classes.start_time, studies_classes.end_time
    FROM studies_classes
GO
```

'studies_events' - BT

Pokazuje wszystkie wydarzenia ze studiów wraz z ich rodzajami (Stacjonarny/Online)

```
CREATE VIEW [dbo].[studies_events] -- BT
```

```
AS
```

```
    SELECT class_id, studies_id, dbo.fn_get_study_class_type(sc.class_id) as 'class_type',  
           title, teacher_id, start_time, end_time from studies_classes sc
```

```
GO
```

'presence_on_finished_events' - BT

Wypisuje listy obecności na ukończonych już wydarzeniach

```
CREATE VIEW [dbo].[presence_on_finished_events] AS -- BT
    SELECT service_id, service_type_name, rodzaj_uslugi, customer_id,
        dbo.fn_get_presence(customer_id,
            service_id, service_type_id) as 'was_present'
    from dbo.event_participants
    WHERE ( service_type_id = 2 OR service_type_id = 4 )
GO
```

'event_participants' - BT

Wypisuje wszystkie usługi wraz z zapisanymi na nie klientami (id klienta) oraz rodzajem usługi

```
CREATE VIEW [dbo].[event_participants] -- BT
AS
    SELECT c.customer_id, e.service_id, e.service_type, e.[online/stationary], e.price,
           e.title, e.start_time, e.end_time,
           dbo.fn_has_access(c.customer_id, e.service_id, st.service_type_id) as
'has_access'
    FROM customers c
    INNER JOIN orders ON c.customer_id = orders.customer_id
    INNER JOIN order_details ON orders.order_id = order_details.order_id
    INNER JOIN dbo.service_types st on order_details.service_type_id = st.service_type_id
    INNER JOIN dbo.events e ON order_details.service_id = e.service_id AND
        st.service_type_name = e.service_type
GO
```

'profit_by_service_type' - BT

Wypisuje przychody dla każdego rodzaju usługi.

```
CREATE VIEW [dbo].[profit_by_service_type]
AS -- BT
    SELECT od.service_type_id, st.service_type_name,
           SUM(dbo.fn_get_price_of_service(od.service_id, od.service_type_id)) AS 'Zysk'
    FROM order_details od
    INNER JOIN service_types st on od.service_type_id = st.service_type_id
    GROUP BY od.service_type_id, st.service_type_name
GO
```

'absence_list_courses' - JM

Zwraca listę wszystkich osób, które są zapisane na kurs i powinny być obecne za zajęciach

```
CREATE VIEW [dbo].[absence_list_courses]
AS
SELECT
dbo.courses.course_id, dbo.users.last_name, dbo.users.first_name, dbo.courses.title
FROM      dbo.customers INNER JOIN
           dbo.users ON dbo.customers.user_id = dbo.users.user_id
           INNER JOIN dbo.orders
           ON dbo.customers.customer_id = dbo.orders.customer_id
           INNER JOIN dbo.order_details
           ON dbo.orders.order_id = dbo.order_details.order_id
           INNER JOIN dbo.courses
           ON dbo.order_details.service_id = dbo.courses.course_id
GO
```

'absence_list_studies_classes' - JM

Zwraca listę wszystkich osób, które są zapisane na studia i powinny być obecne za zajęciach

```
CREATE VIEW [dbo].[absence_list_studies_classes]
AS
SELECT
dbo.studies_classes.title, dbo.studies_classes.studies_id, dbo.studies_classes.class_id,
dbo.users.last_name, dbo.users.first_name
FROM      dbo.orders INNER JOIN
           dbo.order_details ON dbo.orders.order_id = dbo.order_details.order_id
           INNER JOIN
           dbo.customers ON dbo.orders.customer_id = dbo.customers.customer_id
           INNER JOIN
           dbo.studies_classes
           ON dbo.order_details.service_id = dbo.studies_classes.class_id
           INNER JOIN dbo.studies
           ON dbo.studies_classes.studies_id = dbo.studies.studies_id
           INNER JOIN
           dbo.users ON dbo.customers.user_id = dbo.users.user_id
GO
```

'bilocation_raport' - BT

Zwraca listę zajęć które dla danej osoby kolidują z sobą czasowo

```
CREATE VIEW [dbo].[bilocation_raport] AS -- BT
SELECT      t1.id_klienta AS 'id_klienta',
            t1.id_modulu_nadrzednego AS 'id_modulu_nadrzednego_1',
            t1.nazwa_typu_serwisu AS 'nazwa_typu_serwisu_1',
            t1.tytul_zajec AS 'tytul_zajec_1',
            t1.start_time AS 'poczatek_modulu_1',
            t1.end_time AS 'koniec_modulu_1',
            t2.id_modulu_nadrzednego AS 'id_modulu_nadrzednego_2',
            t2.nazwa_typu_serwisu AS 'nazwa_typu_serwisu_2',
            t2.tytul_zajec AS 'tytul_zajec_2',
            t2.start_time AS 'poczatek_modulu_2',
            t2.end_time AS 'koniec_modulu_2'
FROM        time_table t1
INNER JOIN  time_table t2 ON t1.id_klienta = t2.id_klienta
AND        (
            (t1.id_modulu_nadrzednego = t2.id_modulu_nadrzednego
            AND      t1.nazwa_typu_serwisu >= t2.nazwa_typu_serwisu)
            OR
            t1.id_modulu_nadrzednego > t2.id_modulu_nadrzednego
            )
WHERE      (
            -- check if we have the same client_id
            t1.id_klienta = t2.id_klienta
            AND
            (
            -- check if the service is different (any part of it)
            t1.id_modulu_nadrzednego != t2.id_modulu_nadrzednego
            OR      t1.id_zajec != t2.id_zajec
            OR      t1.nazwa_typu_serwisu != t2.nazwa_typu_serwisu
            )
            AND
            (
            dbo.fn_two_datetimes_intersect (
            t1.start_time,
            t1.end_time,
            t2.start_time,
            t2.end_time
            ) = 1
            )
            )
GO
```

'debtors' - JM

Zwraca listę wszystkich dłużników, którzy nie zapłacili w pełni za swoje zamówienie

```
CREATE VIEW [dbo].[debtors]
AS
SELECT dbo.customers.customer_id, SUM(dbo.pending_payments.payment_amount) AS Debt
FROM    dbo.customers INNER JOIN
        dbo.orders ON dbo.customers.customer_id = dbo.orders.customer_id
        INNER JOIN
        dbo.pending_payments
        ON dbo.orders.order_id = dbo.pending_payments.order_id
GROUP BY dbo.customers.customer_id
GO
```

'orders_for_user' - JM

Zwraca listę wszystkich produktów w danym zamówieniu

```
CREATE VIEW [dbo].[orders_for_user]
AS
    SELECT
        c.customer_id, od.order_id, od.service_id, od.service_type_id, st.service_type_name
    FROM order_details od
    INNER JOIN service_types st ON st.service_type_id = od.service_type_id
    INNER JOIN orders o ON od.order_id = o.order_id
    INNER JOIN customers c ON o.customer_id = c.customer_id
GO
```

'price_courses_orders' - JM

Zwraca listę wszystkich kursów wraz z ich cenami w jakim zamówieniu się pojawiło i kto je kupił

```
CREATE VIEW [dbo].[price_courses_orders]
AS
SELECT
dbo.customers.customer_id, dbo.orders.order_id, dbo.courses.price, dbo.courses.title
FROM      dbo.customers INNER JOIN
           dbo.orders ON dbo.customers.customer_id = dbo.orders.customer_id
           INNER JOIN
           dbo.order_details ON dbo.orders.order_id = dbo.order_details.order_id
           INNER JOIN
           dbo.courses ON dbo.order_details.service_id = dbo.courses.course_id
GO
```

'price_studies_orders' - JM

Zwraca listę wszystkich studiów wraz z ich cenami w jakim zamówieniu się pojawiło i kto je kupił

```
CREATE VIEW [dbo].[price_studies_orders]
AS
SELECT
dbo.customers.customer_id, dbo.orders.order_id,
dbo.studies_price_per_classes.member_price,
dbo.studies_price_per_classes.non_member_price,
dbo.studies_classes.title, dbo.studies_classes.class_id,
        dbo.studies_classes.studies_id
FROM      dbo.studies_price_per_classes INNER JOIN
        dbo.studies
        ON dbo.studies_price_per_classes.studies_id = dbo.studies.studies_id
        INNER JOIN dbo.studies_classes
        ON dbo.studies.studies_id = dbo.studies_classes.studies_id INNER JOIN
        dbo.customers INNER JOIN dbo.orders
        ON dbo.customers.customer_id = dbo.orders.customer_id INNER JOIN
        dbo.order_details ON dbo.orders.order_id = dbo.order_details.order_id
        ON dbo.studies_classes.class_id = dbo.order_details.service_id

GO
```

'price_webinars_orders' - JM

Zwraca listę wszystkich webinarów wraz z ich cenami w jakim zamówieniu się pojawiło i kto je kupił

```
CREATE VIEW [dbo].[price_webinars_orders]
AS
SELECT dbo.customers.customer_id, dbo.webinars.webinar_id,
dbo.webinars.title, dbo.webinars.price
FROM      dbo.customers INNER JOIN
           dbo.orders ON dbo.customers.customer_id = dbo.orders.customer_id
           INNER JOIN dbo.order_details
           ON dbo.orders.order_id = dbo.order_details.order_id
           INNER JOIN dbo.courses
           ON dbo.order_details.service_id = dbo.courses.course_id
           INNER JOIN dbo.course_modules
           ON dbo.courses.course_id = dbo.course_modules.course_id
           INNER JOIN dbo.webinars
           ON dbo.order_details.service_id = dbo.webinars.webinar_id
GO
```

'price_for_customers_orders' - JM

Zwraca listę wszystkich zakupów dla danego klienta i jego zamówienia

```
CREATE VIEW [dbo].[price_for_customers_order]
AS
SELECT dbo.customers.customer_id, dbo.orders.order_id,
SUM(dbo.price_studies_orders.member_price)
AS Per_module_for_member, SUM(dbo.price_studies_orders.non_member_price)
AS Per_module_for_nonmember,
SUM(dbo.price_courses_orders.price)
AS [for courses], SUM(dbo.price_webinars_orders.price) AS [for webinars]
FROM      dbo.customers INNER JOIN
          dbo.price_courses_orders
          ON dbo.customers.customer_id = dbo.price_courses_orders.customer_id
          INNER JOIN dbo.price_studies_orders
          ON dbo.customers.customer_id = dbo.price_studies_orders.customer_id
          INNER JOIN dbo.price_webinars_orders
          ON dbo.customers.customer_id = dbo.price_webinars_orders.customer_id
          INNER JOIN dbo.orders
          ON dbo.customers.customer_id = dbo.orders.customer_id
GROUP BY dbo.orders.order_id, dbo.customers.customer_id
GO
```

'shipping_details' - JM

Zwraca listę wszystkich dyplomów wraz z adresem na jaki powinien zostać wysłany dyplom oraz dla kogo on jest

```
CREATE VIEW [dbo].[shipping_details]
AS
SELECT dbo.customers.customer_id, dbo.adress_details.user_id, dbo.users.first_name,
dbo.users.last_name, dbo.adress_details.street, dbo.adress_details.number,
dbo.adress_details.zip, dbo.adress_details.city, dbo.adress_details.country,
        dbo.studies.name, dbo.studies.studies_id AS ID,
dbo.fn_is_passed(dbo.studies.studies_id, 3, dbo.customers.customer_id) AS 'is_passed'
FROM      dbo.adress_details INNER JOIN
        dbo.users ON dbo.adress_details.user_id = dbo.users.user_id INNER JOIN
        dbo.customers ON dbo.users.user_id = dbo.customers.user_id INNER JOIN
        dbo.orders ON dbo.customers.customer_id = dbo.orders.customer_id INNER
JOIN
        dbo.order_details ON dbo.orders.order_id = dbo.order_details.order_id
INNER JOIN
        dbo.studies_classes ON dbo.order_details.service_id =
dbo.studies_classes.class_id INNER JOIN
        dbo.studies ON dbo.studies_classes.studies_id = dbo.studies.studies_id
UNION
SELECT dbo.customers.customer_id, dbo.adress_details.user_id, dbo.users.first_name,
dbo.users.last_name, dbo.adress_details.street, dbo.adress_details.number,
dbo.adress_details.zip, dbo.adress_details.city, dbo.adress_details.country,
        dbo.courses.title, dbo.courses.course_id,
dbo.fn_is_passed(dbo.courses.course_id, 2, dbo.customers.customer_id) AS 'is_passed'
FROM      dbo.adress_details INNER JOIN
        dbo.users ON dbo.adress_details.user_id = dbo.users.user_id INNER JOIN
        dbo.customers ON dbo.users.user_id = dbo.customers.user_id INNER JOIN
        dbo.orders ON dbo.customers.customer_id = dbo.orders.customer_id INNER
JOIN
        dbo.order_details ON dbo.orders.order_id = dbo.order_details.order_id
INNER JOIN
        dbo.courses ON dbo.order_details.service_id = dbo.courses.course_id
UNION
SELECT dbo.customers.customer_id, dbo.adress_details.user_id, dbo.users.first_name,
dbo.users.last_name, dbo.adress_details.street, dbo.adress_details.number,
dbo.adress_details.zip, dbo.adress_details.city, dbo.adress_details.country,
        dbo.webinars.title, dbo.webinars.webinar_id,
dbo.fn_is_passed(dbo.webinars.webinar_id, 1, dbo.customers.customer_id) AS 'is_passed'
FROM      dbo.adress_details INNER JOIN
        dbo.users ON dbo.adress_details.user_id = dbo.users.user_id INNER JOIN
        dbo.customers ON dbo.users.user_id = dbo.customers.user_id INNER JOIN
        dbo.orders ON dbo.customers.customer_id = dbo.orders.customer_id INNER
JOIN
        dbo.order_details ON dbo.orders.order_id = dbo.order_details.order_id
INNER JOIN
```

```
dbo.webinars ON dbo.order_details.service_id = dbo.webinars.webinar_id
```

GO

'table_studies' - JM

Zwraca listę wszystkich studiów wraz z ich spotkaniami godzinami, kiedy się odbywają

```
CREATE VIEW [dbo].[table_studies] AS
SELECT    dbo.studies.studies_id,
          dbo.studies.name,
          dbo.studies_classes.class_id,
          dbo.studies_classes.title,
          dbo.studies_classes.start_time,
          dbo.studies_classes.end_time
FROM      dbo.studies
INNER JOIN dbo.studies_classes
ON        dbo.studies.studies_id = dbo.studies_classes.studies_id
GO
```

'table_webinars' - JM

Zwraca listę wszystkich webinarów wraz z ich spotkaniami godzinami, kiedy się odbywają

```
CREATE VIEW [dbo].[table_webinars] AS
SELECT
    dbo.webinars.webinar_id,
    dbo.webinars.title,
    dbo.webinars.start_time,
    dbo.webinars.end_time,
    dbo.webinars.subject_name,
    dbo.webinar_types.webinar_type
FROM
    dbo.webinars
INNER JOIN
    dbo.webinar_types
ON
    dbo.webinars.webinar_type_id = dbo.webinar_types.webinar_type_id
GO
```

'table_courses' - JM

Zwraca listę wszystkich kursów wraz z ich spotkaniami godzinami, kiedy się odbywają

```
CREATE VIEW [dbo].[table_courses] AS
SELECT    dbo.course_types.course_type,
          dbo.courses.course_id,
          dbo.course_modules.module_id,
          dbo.course_modules.start_time,
          dbo.course_modules.end_time,
          dbo.course_modules.module_title,
          dbo.courses.subject_name,
          dbo.courses.title
FROM      dbo.courses
INNER JOIN dbo.course_modules
ON        dbo.courses.course_id = dbo.course_modules.course_id
INNER JOIN dbo.course_types
ON        dbo.courses.course_type_id = dbo.course_types.course_type_id
GO
```

'table_practicies' - JM

Zwraca listę wszystkich praktyk wraz z datami ich odbywania się jak i nazwą firmy która je organizuje

```
CREATE VIEW [dbo].[table_practices] AS
SELECT    dbo.studies.studies_id,
          dbo.studies.name,
          dbo.studies_practice_modules.practice_id,
          dbo.studies_practice_modules.start_date,
          dbo.studies_practice_modules.end_date,
          dbo.studies_practice_modules.company_name
FROM      dbo.studies_practice_modules
INNER JOIN  dbo.studies
ON  dbo.studies_practice_modules.studies_id = dbo.studies.studies_id
GO
```

'time_table' - JM,BT

Zwraca listę wszystkich serwisów wraz z godzinami ich odbywania się i datami, tworząc kompletny plan dla danego studenta [notatka boczna – dla serwisu typu 'practice' "tytuły zajęć" oznacza nazwę firmy w której są praktyki, a id_zajęć to jest id_pratkyk i id_modulu_nadrzednego jako id_studiów]

```
CREATE VIEW [dbo].[time_table] AS
SELECT o.customer_id AS 'id_klienta',
       course_id AS 'id_modulu_nadrzednego',
       module_id AS 'id_zajec',
       st.service_type_name AS 'nazwa_typu_serwisu',
       module_title AS 'tytul_zajec',
       start_time,
       end_time
FROM   course_modules
INNER JOIN order_details od ON od.service_id = course_id
INNER JOIN orders o ON od.order_id = o.order_id
INNER JOIN service_types st ON st.service_type_id = od.service_type_id
WHERE  st.service_type_name = 'course'
UNION
SELECT o.customer_id,
       webinar_id,
       webinar_id,
       st.service_type_name,
       title,
       start_time,
       end_time
FROM   webinars
INNER JOIN order_details od ON od.service_id = webinars.webinar_id
INNER JOIN orders o ON od.order_id = o.order_id
INNER JOIN service_types st ON st.service_type_id = od.service_type_id
WHERE  st.service_type_name = 'webinar'
UNION
SELECT o.customer_id,
       studies_id,
       class_id,
       st.service_type_name,
       title,
       start_time,
       end_time
FROM   studies_classes
INNER JOIN order_details od ON od.service_id = studies_id
INNER JOIN orders o ON od.order_id = o.order_id
INNER JOIN service_types st ON st.service_type_id = od.service_type_id
WHERE  st.service_type_name = 'studies_classes'
OR     st.service_type_name = 'studies'
UNION
SELECT o.customer_id,
```

```
        studies_id,  
        practice_id,  
        'practices',  
        company_name,  
        start_date,  
        end_date  
FROM    studies_practice_modules  
INNER   JOIN order_details od ON od.service_id = studies_id  
INNER   JOIN orders o ON od.order_id = o.order_id  
GO
```

'staff_time_table' - BT

Zwraca listę wszystkich serwisów wraz z godzinami ich odbywania się i datami, tworząc kompletny plan dla danego nauczyciela oraz tłumacza.

```
CREATE VIEW [dbo].[staff_time_table] AS
    SELECT webinars.webinar_id, webinar_type, 'webinar' as 'type', title,
           teacher_id, translator_id, start_time, end_time
    FROM webinars
    INNER JOIN webinar_types ON webinars.webinar_type_id = webinar_types.webinar_type_id
    LEFT JOIN webinars_transaltions ON webinars.webinar_id =
webinars_transaltions.webinar_id
    UNION
    SELECT course_modules.module_id, module_type_name, 'courses' as 'type', module_title,
           teacher_id, translator_id, start_time, end_time
    FROM course_modules
    INNER JOIN course_module_types ON
           course_module_types.module_type_id = course_modules.module_type_id
    LEFT JOIN course_translations ON
           course_modules.module_id = course_translations.module_id
    UNION
    SELECT studies_classes.class_id,
dbo.fn_get_study_class_type(studies_classes.class_id),
           'studies_classes' as 'type', title, teacher_id, translator_id, start_time,
end_time
    FROM studies_classes
    LEFT JOIN studies_class_translators ON
           studies_classes.class_id = studies_class_translators.class_id
GO
```

Typy danych

'SERVICES_TABLE_TYPE' - RM

Typ danych wykorzystywany podczas dodawania produktów w 'order_details'.

```
CREATE TYPE SERVICES_TABLE_TYPE AS TABLE (  
    service_id INT,  
    service_type_id INT  
)
```

'INSTALMENTS_TABLE_TYPE' - RM

Typ danych wykorzystywany przy podziale zamówienia na raty lub odroczone płatności.

```
CREATE TYPE INSTALMENTS_TABLE_TYPE AS TABLE (  
    instalment_id INT IDENTITY(0,1),  
    instalment MONEY,  
    payment_due DATETIME,  
    payment_link VARCHAR(MAX)  
)
```

Funkcje

'fn_sum_services_table' - RM

Sumuję całkowitą cenę zamówień w typie tabelowym 'SERVICES_TABLE_TYPE'.

```
CREATE FUNCTION fn_sum_services_table(@services_table SERVICES_TABLE_TYPE READONLY)
RETURNS money AS
BEGIN
    DECLARE @totalVal money
    SET @totalVal = (
        SELECT SUM([dbo].[fn_get_price_of_service](service_id, service_type_id))
        FROM @services_table
    )
    RETURN @totalVal
END
```

'fn_has_access(@customer_id INT, @service_id INT, @service_type_id INT)' - RM

Zwraca BIT informujący czy użytkownik ma dostęp dla danego serwisu.

```
CREATE FUNCTION fn_has_access(@customer_id INT, @service_id INT, @service_type_id INT)
RETURNS BIT AS
BEGIN
    DECLARE @has_access BIT;
    SET @has_access = 0;

    /* Sprawdź czy klient, kupował ten produkt. */
    SET @has_access = IIF(EXISTS(
        SELECT customer_id, service_id, service_type_id
        FROM user_completed_payment_details
        WHERE customer_id=@customer_id AND service_id=@service_id
        AND service_type_id=@service_type_id
        UNION
        SELECT customer_id, service_id, service_type_id
        FROM user_pending_payment_details
        WHERE customer_id=@customer_id AND service_id=@service_id
        AND service_type_id=@service_type_id), 1, 0)

    /* Jeśli nie kupował to nie ma dostępu. */
    IF @has_access=0
        RETURN 0;

    /* Sprawdź czy nie zalega z oczekującymi opłatami. */
    SET @has_access = IIF(EXISTS(SELECT * FROM dbo.user_pending_payment_details
        WHERE customer_id=@customer_id AND service_id=@service_id
        AND service_type_id=@service_type_id AND GETDATE()>payment_due), 0, 1)

    RETURN @has_access;
END
```

'fn_calc_enrollment(@service_id INT, @service_type_id INT)' - RM

Zwraca liczbę, ile jest zapisów na dany serwis.

```
CREATE FUNCTION fn_calc_enrollment(@service_id INT, @service_type_id INT)
RETURNS INT AS
BEGIN
    RETURN (
        COUNT customer_id FROM dbo.event_participants ep
        WHERE ep.service_id=@service_id AND (
            SELECT service_type_id FROM service_types st
            WHERE st.service_type_name=ep.service_type_name
            )=@service_type_id
        )
    )
END
```

'fn_get_deadline(@service_id INT, @service_type_id INT)' - RM

Zwraca ostatnią datę, do której przyjmujemy zamówienia.

```
CREATE FUNCTION fn_get_deadline(@service_id INT, @service_type_id INT)
RETURNS DATETIME AS
BEGIN
    RETURN (
        SELECT purchase_deadline FROM dbo.events_purchase_deadline epd
        WHERE epd.service_id=@service_id AND epd.service_type_id=@service_type_id
        )
END
```

'fn_calc_limit_left(@service_id INT, @service_type_id INT)' - RM

Zwraca ile miejsc do zapisu pozostało.

```
CREATE FUNCTION fn_calc_limit_left(@service_id INT, @service_type_id INT)
RETURNS INT AS
BEGIN
    DECLARE @limit_left INT = (
        SELECT (limit - dbo.fn_calc_enrollment(@service_id, @service_type_id))
        FROM dbo.events_purchase_deadline epd
        WHERE epd.service_id=@service_id AND epd.service_type_id=@service_type_id
    );
    IF @limit_left IS NULL
        RETURN NULL;
    RETURN IIF(@limit_left>0, @limit_left, 0);
END
```

'fn_is_passed(@customer_id INT, @service_id INT, @service_type_id INT)' - RM

Funkcja zwraca BIT informujący czy dany użytkownik zaliczył dany kurs/studia (lub był obecny w innym przypadku).

```
CREATE FUNCTION fn_is_passed(@customer_id INT, @service_id INT, @service_type_id INT)
RETURNS BIT AS
BEGIN
```

```
    DECLARE @is_passed BIT = CASE

        -- webinar
        WHEN @service_type_id=1 THEN IIF(EXISTS (
            SELECT *
            FROM dbo.presences_view pv
            WHERE pv.event_id=@service_id
            AND pv.event_name='webinar'
            AND pv.customer_id=@customer_id
            AND pv.was_present=1
        ), 1, 0)

        -- kursy: 80% modułów
        WHEN @service_type_id=2 THEN IIF(
            (
                SELECT COUNT(*)
                FROM dbo.presences_view pv
                WHERE pv.event_id=@service_id
                AND pv.event_name='course'
                AND pv.customer_id=@customer_id
                AND pv.was_present=1
            ) /
            NULLIF((
                SELECT COUNT(*)
                FROM dbo.presences_view pv
                WHERE pv.event_id=@service_id
                AND pv.event_name='course'
                AND pv.customer_id=@customer_id), 0)
            >= 0.8, 1, 0)

        -- studia: 80% ćwiczeń + zaliczone praktyki
        WHEN @service_type_id=3 THEN IIF(
            (
                SELECT COUNT(*)
                FROM dbo.presences_view pv
                WHERE pv.event_id=@service_id
                AND pv.event_name='studies_class'
                AND pv.customer_id=@customer_id
```

```

AND pv.was_present=1
) /
NULLIF((
SELECT COUNT(*)
FROM dbo.presences_view pv
WHERE pv.event_id=@service_id
AND pv.event_name='studies_class'
AND pv.customer_id=@customer_id), 0) >= 0.8
AND NOT EXISTS (
    SELECT *
    FROM dbo.presences_view pv
    WHERE pv.event_id=@service_id
    AND pv.event_name='practice'
    AND pv.customer_id=@customer_id
    AND pv.was_present=0
)
AND EXISTS (
    SELECT *
    FROM dbo.presences_view pv
    WHERE pv.event_id=@service_id
    AND pv.event_name='practice'
    AND pv.customer_id=@customer_id
    AND pv.was_present=1
)
, 1, 0)

```

-- pojedyncze spotkanie na studiach: obecność

```

WHEN @service_type_id=4 THEN IIF(EXISTS (
    SELECT *
    FROM dbo.presences_view pv
    WHERE pv.event_id=@service_id
    AND pv.event_name='studies_class'
    AND pv.customer_id=@customer_id
    AND pv.was_present=1
), 1, 0)

```

ELSE 0

END;

RETURN @is_passed;

END

GO

fn_calc_total_order_value(@order_id) BT

zwraca całkowitą wartość zamówienia o danym id

```
CREATE FUNCTION [dbo].[fn_calc_total_order_value](@order_id int)
```

```
RETURNS money AS
```

```
BEGIN
```

```
    DECLARE @totalVal money;
```

```
    SET @totalVal = (
```

```
        SELECT SUM(dbo.fn_get_price_of_service(service_id, service_type_id))
```

```
        FROM order_details
```

```
        WHERE order_id = @order_id
```

```
    )
```

```
    RETURN @totalVal;
```

```
END
```

```
GO
```

fn_staff_has_free_time(@employee_id, @start_time, @end_time) BT

Zwraca 1 jeśli nauczyciel/tłumacz którego id podaliśmy nie ma zagospodarowanego czasu w przedziale (start_time-end_time), czyli ma wolny grafik.

```
-- Function to check if staff isn't busy
CREATE FUNCTION [dbo].[fn_staff_has_free_time](
    @employee_id int,
    @start_time datetime,
    @end_time datetime
)
RETURNS BIT AS
BEGIN
    DECLARE @has_free_time bit;
    SET @has_free_time = (
        SELECT MAX(CAST(dbo.fn_two_datetimes_intersect(@start_time, @end_time,
            start_time, end_time) AS INT))
        FROM staff_time_table
        WHERE teacher_id = @employee_id OR translator_id = @employee_id
    )
    RETURN @has_free_time;
END
GO
```

fn_get_price_of_service(@service_id, @service_type_id) BT

zwraca cenę danej usługi.

```
CREATE FUNCTION [dbo].[fn_get_price_of_service](@id int, @type_id int)
RETURNS money AS
BEGIN
    DECLARE @price money;
    SELECT @price = (
        CASE
            WHEN @type_id = 1 THEN (
                SELECT price
                FROM webinars
                WHERE webinars.webinar_id = @id
            )
            WHEN @type_id = 2 THEN (
                SELECT price
                FROM courses
                WHERE courses.course_id = @id
            )
            WHEN @type_id = 3 THEN (
                SELECT
                    pr.member_price * dbo.fn_get_number_of_college_classes(pr.studies_id)
                FROM studies
                JOIN dbo.studies_price_per_classes pr
                ON studies.studies_id = pr.studies_id
                WHERE studies.studies_id = @id
            )
            WHEN @type_id = 4 THEN (
                SELECT pr.non_member_price
                FROM studies
                JOIN dbo.studies_price_per_classes pr
                ON studies.studies_id = pr.studies_id
                WHERE studies.studies_id = @id
            )
        )
    END
);

RETURN @price;
END;
GO
```

fn_get_presence(@customer_id, @service_id, @service_type_id) BT

Zwraca 1, jeśli klient był na danych zajęciach, 0 w przeciwnym wypadku

```
CREATE FUNCTION [dbo].[fn_get_presence](@customer_id int, @service_id int,
@service_type_id int) -- BT
RETURNS bit AS
BEGIN
    DECLARE @was_present bit;

    SET @was_present = CASE
        WHEN @service_type_id = 1 THEN (SELECT 1) -- pointless for this argument
        WHEN @service_type_id = 2 THEN (
            IIF(EXISTS(SELECT customer_id FROM course_presences
                WHERE customer_id = @customer_id AND module_id = @service_id), 1,
0)
        )
        WHEN @service_type_id = 3 THEN (SELECT 1) -- pointless for this argument
        WHEN @service_type_id = 4 THEN (
            IIF(EXISTS(SELECT customer_id FROM studies_presences
                WHERE customer_id = @customer_id AND class_id = @service_id), 1, 0)
        )
    END

    RETURN @was_present
END
GO
```

fn_get_service_type(@service_id, @service_type_id)

Zwraca rodzaj całej usługi (webinarium, cały kurs, całe studia, pojedyncze spotkanie)

```
CREATE FUNCTION [dbo].[fn_get_service_type](@service_id int, @service_type_id int) -- BT
RETURNS nvarchar(MAX) AS
BEGIN
    DECLARE @service_type nvarchar(MAX);
    SET @service_type = CASE
        WHEN @service_type_id = 2 THEN (
            SELECT course_type FROM course_types
            WHERE course_type_id = (SELECT course_type_id FROM courses
                                   WHERE course_id = @service_id)
        )
        ELSE
            dbo.fn_get_meeting_type(@service_id, @service_type_id)
        END
    RETURN @service_type
end
GO
```

fn_get_meeting_type(@service_id, @service_type_id) BT

Zwraca rodzaj pojedynczego spotkania dla danej usługi. Jesli wybralismy studia, zwraca 'Hybrydowe'

```
CREATE FUNCTION [dbo].[fn_get_meeting_type](@service_id int, @service_type_id int) -- BT
RETURNS nvarchar(MAX) AS
BEGIN
    DECLARE @meeting_type nvarchar(MAX);
    SET @meeting_type = CASE
        WHEN @service_type_id = 1 THEN (
            SELECT webinar_type FROM dbo.webinar_types WHERE webinar_type_id = (
                SELECT webinar_type_id FROM dbo.webinars WHERE webinar_id =
@service_id)
            )
        WHEN @service_type_id = 2 THEN (
            SELECT module_type_name FROM course_module_types WHERE module_type_id = (
                SELECT module_type_id FROM course_modules WHERE course_id =
@service_id)
            )
        WHEN @service_type_id = 3 THEN (
            SELECT 'Hybrid'
            )
        WHEN @service_type_id = 4 THEN (
            SELECT dbo.fn_get_study_class_type(@service_id)
            )
        end
    RETURN @meeting_type
end
GO
```

fn_get_study_class_type(@class_id) - BT
Zwraca rodzaj zajęć na uczelni (Stationary/Online)

```
CREATE FUNCTION [dbo].[fn_get_study_class_type](@class_id int) --BT
RETURNS nvarchar(MAX) AS
BEGIN
    DECLARE @class_type nvarchar(MAX);
    IF EXISTS (SELECT class_id FROM studies_online_classes WHERE class_id = @class_id)
    BEGIN
        SET @class_type = 'Online';
    end
    ELSE
    BEGIN
        SET @class_type = 'Stationary'
    end
    RETURN @class_type;
end
GO
```

fn_get_number_of_college_classes(@studies_id) BT

zwraca ilość zajęć na studiach o danym id.

```
CREATE FUNCTION [dbo].[fn_get_number_of_college_classes](@studies_id int)
RETURNS int AS
BEGIN
    DECLARE @numberOfClasses int;
    SET @numberOfClasses = (SELECT COUNT(*) FROM dbo.studies_classes
                           WHERE studies_id = @studies_id);
    RETURN @numberOfClasses;
end
GO
```

fn_calc_average_order_value() BT

zwraca średnią wartość zamówienia (przed obniżkami)

```
CREATE FUNCTION [dbo].[fn_calc_average_order_value]()
RETURNS money AS
BEGIN
    DECLARE @average money;
    SET @average = (
        SELECT AVG(dbo.fn_calc_total_order_value(order_id))
        FROM orders
    );
    RETURN @average;
end
GO
```

fn_get_most_expensive_order_price() BT

zwraca cenę najdroższego zamówienia (przed obniżkami)

```
CREATE FUNCTION [dbo].[fn_get_most_expensive_order_price]()
RETURNS money AS
BEGIN
    DECLARE @maximum money;
    SET @maximum = (
        SELECT MAX(dbo.fn_calc_total_order_value(order_id))
        FROM orders
    );
    RETURN @maximum;
end
GO
```

fn_get_least_expensive_order_price() BT

zwraca cenę najtańszego zamówienia (przed obniżkami)

```
CREATE FUNCTION [dbo].[fn_get_least_expensive_order_price]()
RETURNS money AS
BEGIN
    DECLARE @minimum money;
    SET @minimum = (
        SELECT MIN(dbo.fn_calc_total_order_value(order_id))
        FROM orders
    );
    RETURN @minimum;
end
GO
```

'fn_check_completed_payment' - RM

Sprawdza czy dla danego klienta płatność za dany produkt została ukończona.

```
CREATE FUNCTION [dbo].[fn_check_completed_payment](@customer_id INT, @service_id INT,
@service_type_id INT)
RETURNS BIT
AS
BEGIN
    DECLARE @result BIT

    IF EXISTS (SELECT * FROM user_completed_payment_details WHERE customer_id=@customer_id
AND service_id=@service_id AND service_type_id=@service_type_id)
        SET @result = 1;
    ELSE
        SET @result = 0;

    RETURN @result
END
GO
```

'fn_check_pending_payment' - RM

Sprawdza czy dla danego klienta płatność za dany produkt jest oczekująca.

```
CREATE FUNCTION fn_check_pending_payment(@customer_id INT, @service_id INT,  
@service_type_id INT)  
RETURNS BIT  
AS  
BEGIN  
    DECLARE @result BIT  
  
    IF EXISTS (SELECT * FROM user_pending_payment_details WHERE customer_id=@customer_id  
AND service_id=@service_id AND service_type_id=@service_type_id)  
        SET @result = 1;  
    ELSE  
        SET @result = 0;  
  
    RETURN @result  
END  
GO
```

'fn_sum_services_table' - RM

Policz wartość zamówienia dla tabeli produktów.

```
CREATE FUNCTION fn_sum_services_table(@services_table SERVICES_TABLE_TYPE READONLY)
RETURNS money AS
BEGIN
    DECLARE @totalVal money
    SET @totalVal = (
        SELECT SUM([dbo].[fn_get_price_of_service](service_id, service_type_id))
        FROM @services_table
    )
    RETURN @totalVal
END
```

'fn_get_bilocation_for_user' - BT

Zwraca raport bilokacji dla użytkownika o danym id

```
CREATE FUNCTION [dbo].[fn_get_bilocation_for_user](@user_id int)
RETURNS TABLE AS RETURN
    SELECT * from bilocation_raport
    where id_klienta = @user_id
GO
```

'fn_get_time_table_for_user' - BT

Zwraca plan zajęć dla użytkownika o danym id

```
CREATE FUNCTION [dbo].[fn_get_time_table_for_user](@user_id int)
RETURNS TABLE AS RETURN
    SELECT * from time_table
    where id_klienta = @user_id
GO
```

'fn_get_debt_for_user' - BT

Zwraca dług użytkownika o danym id

```
CREATE FUNCTION [dbo].[fn_get_debt_for_user](@user_id int)
RETURNS TABLE AS RETURN
    SELECT debtors.customer_id, debtors.Debt from debtors
    INNER JOIN dbo.customers c on debtors.customer_id = c.customer_id
    where @user_id = debtors.customer_id
GO
```

'fn_get_staff_time_table_for_teacher' - BT

Zwraca plan zajęć (pracy) dla nauczyciela o danym id

```
CREATE FUNCTION [dbo].[fn_get_staff_time_table_for_teacher](@teacher_id int)
RETURNS TABLE AS RETURN
    SELECT * from staff_time_table
    WHERE teacher_id=@teacher_id
GO
```

'fn_get_staff_time_table_for_translator' - BT

Zwraca plan zajęć (pracy) dla tłumacza o danym id

```
CREATE FUNCTION [dbo].[fn_get_staff_time_table_for_translator](@translator_id int)
RETURNS TABLE AS RETURN
    SELECT * from staff_time_table
    WHERE teacher_id=@translator_id
GO
```

'fn_get_event_participants_with_access_to' - BT

Zwraca uczestników modułu zajęć o danym id i o danym typie, którzy mają do niego wykupiony dostęp.

```
CREATE FUNCTION [dbo].[fn_get_event_participants_with_access_to](@service_id int,  
@service_type nvarchar(MAX))  
RETURNS TABLE AS RETURN  
    SELECT customer_id,  
           service_id,  
           service_type,  
           [online/stationary],  
           price,  
           title,  
           start_time,  
           end_time  
    from event_participants  
    WHERE service_id=@service_id AND service_type=@service_type AND has_access=1  
GO
```

Procedury

'usp_orders_insert' - RM

Wstawia nowe zamówienie do tabeli 'orders' wraz ze szczegółami w 'order_details' oraz dodaje nadchodzące płatności w 'pending_payments'. Korzysta z wcześniej zdefiniowanego [typu](#) oraz [funkcji](#).

```
CREATE PROCEDURE usp_orders_insert(
    @customer_id INT,
    @discount DECIMAL(3,2) = 0,
    @services_table SERVICES_TABLE_TYPE READONLY,
    @instalments_table INSTALMENTS_TABLE_TYPE READONLY
) AS
BEGIN
    DECLARE @errorMessage NVARCHAR(MAX);
    SET @errorMessage = 'Nie udało dodać się zamówienia';
    BEGIN TRY
        BEGIN TRANSACTION [insert_order]

        DECLARE @order_id INT
        DECLARE @order_time DATETIME
        SET @order_id = (SELECT ISNULL(MAX(order_id), 0) + 1 FROM orders)
        SET @order_time = GETDATE()

        IF NOT EXISTS (SELECT * FROM @services_table)
        BEGIN
            SET @errorMessage = 'Brak order_details';
            THROW 50000, @errorMessage, 1;
        END

        IF EXISTS (
            SELECT @customer_id, service_id, service_type_id FROM @services_table
            INTERSECT
            SELECT customer_id, service_id, service_type_id FROM dbo.orders_for_user)
        BEGIN
            SET @errorMessage = 'Ten użytkownik posiada już któryś z produktów.';
            THROW 50000, @errorMessage, 1;
        END

        /* Sprawdź czy jest przed deadlineem zapisu. */
        DECLARE @min_dealine DATETIME = (
            SELECT MIN(ISNULL(dbo.fn_get_deadline(service_id, service_type_id),0))
            FROM @services_table
        );
        IF GETDATE() > @min_dealine
        BEGIN
            SET @errorMessage = 'Jeden z produktów nie jest już w okresie zapisów.';
            THROW 50000, @errorMessage, 1;
        END
    END TRY
    BEGIN CATCH
        SET @errorMessage = 'Wystąpił błąd podczas zapisu zamówienia.';
        THROW 50000, @errorMessage, 1;
    END CATCH
    ROLLBACK TRANSACTION [insert_order]
END
```

END

/* Sprawdź czy są miejsca. NULL - bez limitu */

```
DECLARE @min_limit DATETIME = (  
    SELECT MIN(dbo.fn_calc_limit_left(service_id, service_type_id))  
    FROM @services_table  
);
```

```
IF @min_limit IS NOT NULL AND @min_limit <= 0
```

```
BEGIN
```

```
    SET @errorMessage = 'Brak wolnych miejsc na jeden z produktów.';
```

```
    THROW 50000, @errorMessage, 1;
```

```
END
```

-- dodaj order

```
INSERT INTO orders (order_id, customer_id, order_time)
```

```
VALUES
```

```
    (@order_id, @customer_id, @order_time)
```

-- dodaj poszczególne pozycje

```
INSERT INTO order_details (order_id, service_id, service_type_id)
```

```
SELECT @order_id, service_id, service_type_id
```

```
FROM @services_table
```

-- dodaj płatność

```
DECLARE @total_price MONEY
```

```
DECLARE @payment_id INT
```

```
SELECT @total_price = [dbo].[fn_sum_services_table](@services_table)
```

```
SELECT @payment_id = ISNULL(MAX(payment_id), 0) + 1 FROM pending_payments
```

-- sprawdź czy raty sumują się do całości zamówienia

```
IF (SELECT SUM(instalment) FROM @instalments_table) != @total_price
```

```
BEGIN
```

```
    SET @errorMessage = 'Nieprawidłowy rozkład na raty';
```

```
    THROW 50000, @errorMessage, 1;
```

```
END
```

```
INSERT INTO pending_payments
```

```
(payment_id, order_id, payment_amount, payment_due, payment_link)
```

```
SELECT
```

```
    @payment_id + instalment_id,
```

```
    @order_id,
```

```
    instalment*(1-@discount),
```

```
    payment_due,
```

```
    payment_link
```

```
FROM @instalments_table
```

```
        COMMIT TRANSACTION [insert_order];
    END TRY

    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION [insert_order];
        THROW 50000, @errorMessage, 1;
    END CATCH

END

GO
```

'usp_delete_pending_order' - RM

Usuwa zamówienie oczekujące na płatność, jeśli nie została opłacona jeszcze żadna rata i użytkownik nie mógł skorzystać z żadnego produktu.

```
ALTER PROCEDURE usp_delete_pending_order(@order_id INT) AS
BEGIN
    DECLARE @errorMessage NVARCHAR(MAX);
    SET @errorMessage = 'Nie udało się usunąć oczekującej płatności.';
    BEGIN TRY
        BEGIN TRANSACTION [delete_order]

        IF NOT EXISTS (SELECT * FROM pending_payments WHERE order_id=@order_id)
        BEGIN
            SET @errorMessage = 'Brak oczekującego zamówienia order_id: '
                + CONVERT(NVARCHAR, @order_id);
            THROW 50000, @errorMessage, 1;
        END

        IF EXISTS (SELECT * FROM pending_payments WHERE order_id=@order_id
            AND GETDATE()>=pending_payments.payment_due)
        BEGIN
            SET @errorMessage = 'Nie można anulować zalegającej płatności order_id: '
                + CONVERT(NVARCHAR, @order_id);
            THROW 50000, @errorMessage, 1;
        END

        /* Sprawdź czy nie jest już po deadline zapisów.
        Użytkownik już mógł skorzystać z jakiegoś serwisu! */
        DECLARE @min_dealine DATETIME = (
            SELECT MIN(ISNULL(dbo.fn_get_deadline(service_id, service_type_id),0))
            FROM order_details od WHERE od.order_id=@order_id
        );
        IF GETDATE() >= @min_dealine
        BEGIN
            SET @errorMessage = 'Za późno na automatyczne anulowanie zamówienia.';
            THROW 50000, @errorMessage, 1;
        END

        IF EXISTS (SELECT * FROM completed_payments WHERE order_id=@order_id)
        BEGIN
            SET @errorMessage =
                'Część zamówienia została opłacona. ' +
                'Należy manualnie rozwiązać problem dla order_id: '
                + CONVERT(NVARCHAR, @order_id);
            THROW 50000, @errorMessage, 1;
        END
    END TRY
    END
```

```
-- usuń zamówienie oczekujące na płatność
DELETE FROM pending_payments WHERE order_id=@order_id

COMMIT TRANSACTION [delete_order];
END TRY

BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION [delete_order];
    THROW 50000, @errorMessage, 1;
END CATCH
END
```

‘usp_complete_payment’ - RM

Po otrzymaniu informacji od zewnętrznego serwisu ta procedura zaktualizuje status płatności w naszej bazie.

```
CREATE PROCEDURE usp_complete_payment(
    @payment_id INT,
    @payment_time DATETIME
)
AS
BEGIN
    DECLARE @errorMessage NVARCHAR(MAX);

    BEGIN TRY
        BEGIN TRANSACTION [complete_payment]

        IF NOT EXISTS (SELECT * FROM pending_payments WHERE payment_id=@payment_id)
        BEGIN
            SET @errorMessage = 'Brak zamówienia dla payment_id: ' +
                CONVERT(NVARCHAR, @payment_id);
            THROW 50000, @errorMessage, 1;
        END

        -- dodaj do completed_payments
        INSERT INTO completed_payments
        (payment_id, order_id, payment_amount, payment_time)
        SELECT @payment_id, order_id, payment_amount, @payment_time
        FROM pending_payments WHERE payment_id=@payment_id

        -- usuń z pending_payments
        DELETE FROM pending_payments WHERE payment_id=@payment_id

        COMMIT TRANSACTION [complete_payment];
    END TRY

    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION [complete_payment];

        SET @errorMessage = 'Nie udało przetworzyć payment_id: ' +
            CONVERT(NVARCHAR, @payment_id);
        THROW 50000, @errorMessage, 1;
    END CATCH
END

GO
```

usp_user_insert' BT

Wstawia nowego użytkownika do tabeli 'users'. Jeśli danej kombinacji miasto + państwo nie ma w tabeli countries_cities, to dodaje ją tam. Jeśli użytkownik jest nauczycielem/tłumaczem to przyporządkowuje mu teacher_id w tabeli 'teachers'. Jeśli jest tłumaczem to również ustawia mu język i id tłumacza w 'translators'.

```
CREATE PROCEDURE [dbo].[usp_users_insert](
    @user_type_id INT,
    @e_mail NVARCHAR(MAX),
    @hashed_password NVARCHAR(MAX),
    @first_name NVARCHAR(MAX),
    @last_name NVARCHAR(MAX),
    @phone_number NVARCHAR(MAX) = NULL,
    @street NVARCHAR(MAX),
    @number int,
    @zip NVARCHAR(MAX),
    @city NVARCHAR(63),
    @country NVARCHAR(63),
    @language_id int = NULL
)
AS BEGIN
    TRY
        DECLARE @user_id int;
        SET @user_id = (SELECT ISNULL(MAX(user_id), 0) + 1 FROM users);

        -- INSERT INTO users TABLE
        INSERT INTO dbo.users(user_id, user_type_id, e_mail, hashed_passwd, first_name,
last_name, phone_number)
        VALUES (@user_id, @user_type_id, @e_mail, @hashed_password, @first_name,
            @last_name, @phone_number)

        -- INSERT INTO adress
        INSERT INTO adress_details(user_id, street, number, zip, city, country)
        VALUES (@user_id, @street, @number, @zip, @city, @country)

        -- INSERT INTO countries_cities if not exists
        IF NOT EXISTS (SELECT * FROM countries_cities
            WHERE country = @country AND city = @city)
        BEGIN
            INSERT INTO countries_cities(country, city)
            VALUES (@country, @city)
        END

        -- CHECK NEW USER TYPE
        DECLARE @user_type_name NVARCHAR(MAX);
        SET @user_type_name = (SELECT user_type_name FROM user_types
            WHERE user_type_id = @user_type_id);
```

```
--      IF WE ARE ADDING A TEACHER
IF @user_type_name = 'Teacher'
BEGIN
    EXEC usp_teacher_insert @user_id
END

--      IF WE ARE ADDING A TRANSLATOR
IF @user_type_name = 'Translator'
BEGIN
    EXEC usp_translator_insert @user_id, @language_id
END
END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION
    PRINT ERROR_MESSAGE()
    RAISERROR ('Cos poszlo nie tak', 10, 1)
END CATCH
```

‘usp_teacher_insert’ BT

Pomocnicza, wstawia użytkownika o danym user_id do tabeli ‘teachers’

```
CREATE PROCEDURE [dbo].[usp_teacher_insert](
    @user_id int
)
AS BEGIN
    TRY
        DECLARE @teacher_id int;
        SET @teacher_id = (SELECT ISNULL(MAX(teacher_id), 0) + 1 FROM teachers);

        -- INSERT INTO TEACHER TABLE
        INSERT INTO teachers(teacher_id, user_id)
        VALUES (@teacher_id, @user_id)
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION
        PRINT ERROR_MESSAGE()
        RAISERROR ('Cos poszlo nie tak', 10, 1)
    END CATCH
```

'usp_translator_insert' BT

Pomocnicza, wstawia użytkownika o danym user_id do tabeli 'teachers' oraz do 'translators' z przekazany językiem

```
CREATE PROCEDURE [dbo].[usp_translator_insert](
    @user_id int,
    @language_id int
)
AS BEGIN
    TRY
        DECLARE @teacher_id int;
        SET @teacher_id = (SELECT ISNULL(MAX(teacher_id), 0) + 1 FROM teachers);

        DECLARE @translator_id int;
        SET @translator_id = (SELECT ISNULL(MAX(translator_id), 0) + 1 FROM translators);

        --      INSERT INTO TEACHER TABLE
        INSERT INTO teachers(teacher_id, user_id)
        VALUES (@teacher_id, @user_id)

        --      INSERT INTO TRANSLATOR TABLE
        INSERT INTO translators(translator_id, teacher_id, language_id)
        VALUES (@translator_id, @teacher_id, @language_id)
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION
        PRINT ERROR_MESSAGE()
        RAISERROR ('Cos poszlo nie tak', 10, 1)
    END CATCH
```

'usp_webinar_insert' BT

Wstawia nowy webinar do tabeli webinars. Jeśli jest podany tłumacz, to wstawia go do tabeli webinars_translations.

```
CREATE PROCEDURE [dbo].[usp_webinar_insert](
    @webinar_type_id int,
    @price money,
    @title nvarchar(MAX),
    @subject_name nvarchar(MAX),
    @teacher_id int,
    @start_time datetime,
    @end_time datetime,
    @translator_id int = NULL,
    @link nvarchar(MAX) = NULL,
    @link_expires datetime = NULL
)
AS BEGIN
    TRY
        -- CHECK IF STAFF'S TIME TABLE DOESN'T COLLIDE WITH WEBINAR
        IF(dbo.fn_staff_has_free_time(
@teacher_id, @start_time, @end_time) = 0 OR
        dbo.fn_staff_has_free_time(
@translator_id, @start_time, @end_time) = 0
        )
            BEGIN
                THROW 50000, N'Nauczyciel/tłumacz jest zajęty w danym terminie', 1
            end

        -- CHECK IF START_TIME IS IN THE FUTURE
        IF(@start_time < getdate())
            BEGIN
                THROW 50000, N'Data startu webinaru już upłynęła.', 1
            end

        IF(@link_expires IS NOT NULL AND @link_expires < getdate())
            BEGIN
                THROW 50000, N'Data wygaśnięcia linku już upłynęła', 1
            end

        DECLARE @webinar_id int;
        SET @webinar_id = (SELECT ISNULL(MAX(webinar_id), 0) + 1 FROM webinars)

        INSERT INTO webinars (webinar_id, webinar_type_id, price, title,
                                subject_name, link, link_expires, teacher_id, start_time,
end_time)
        VALUES (@webinar_id, @webinar_type_id, @price, @title,
                @subject_name, @link, @link_expires, @teacher_id, @start_time, @end_time)
```

```

IF(@translator_id IS NOT NULL)
BEGIN
    INSERT INTO webinars_transaltions(webinar_id, translator_id)
    VALUES (@webinar_id, @translator_id)
end
END TRY
BEGIN CATCH
    IF(@@TRANCOUNT > 0)
        ROLLBACK TRAN
    DECLARE @ErrorMessage NVARCHAR(4000);
    DECLARE @ErrorNumber INT;
    DECLARE @ErrorState INT;

    SET @ErrorNumber = ERROR_NUMBER();
    SET @ErrorMessage = ERROR_MESSAGE();
    SET @ErrorState = ERROR_STATE();

    THROW @ErrorNumber, @ErrorMessage, @ErrorState
END CATCH
GO

```

'usp_course_insert' BT

Wstawia cały kurs do tabeli courses. Przyjmuje JSON z informacjami o liscie modulow kursu, dla kazdego z nich tworzac rekord w course_modules korzystając z procedury usp_course_module_insert.

```
CREATE PROCEDURE [dbo].[usp_course_insert](
    @course_type_id int,
    @title nvarchar(MAX),
    @subject_name nvarchar(MAX),
    @price money,
    @module_list_JSON as nvarchar(MAX)
)
AS
BEGIN
    TRY
        DECLARE @course_id int;
        SET @course_id = (SELECT ISNULL(MAX(course_id), 0) + 1 FROM courses);

        INSERT INTO courses(course_id, course_type_id, title, subject_name, price)
        VALUES (@course_id, @course_type_id, @title, @subject_name, @price)

        --      now we want to add all courses into their corresponding tables
        --      first we need to unpack module list out of json
        DECLARE @module_type_id INT;
        DECLARE @module_title NVARCHAR(MAX);
        DECLARE @teacher_id INT;
        DECLARE @start_time DATETIME;
        DECLARE @end_time DATETIME;
        DECLARE @translator_id INT;
        DECLARE @link NVARCHAR(MAX);
        DECLARE @classroom_id INT;
        DECLARE @module_list Table (
            module_type_id int,
            module_title nvarchar(MAX),
            teacher_id int,
            start_time datetime,
            end_time datetime,
            translator_id int NULL,
            link nvarchar(MAX) NULL,
            classroom_id int NULL
        );

        INSERT INTO @module_list (module_type_id, module_title, teacher_id,
start_time, end_time, translator_id, link, classroom_id)
        SELECT module_type_id, module_title, teacher_id, start_time, end_time,
translator_id, link, classroom_id
        FROM OPENJSON(@module_list_JSON)
        WITH (
```

```

        module_type_id int,
        module_title nvarchar(MAX),
        teacher_id int,
        start_time datetime,
        end_time datetime,
        translator_id int,
        link nvarchar(MAX),
        classroom_id int
    );

--      now create cursor and iterate over modules, adding them with procedure
DECLARE cur_iterator CURSOR FOR
SELECT module_type_id, module_title, teacher_id, start_time,
       end_time, translator_id, link, classroom_id
FROM @module_list;

OPEN cur_iterator;
FETCH NEXT FROM cur_iterator INTO @module_type_id, @module_title, @teacher_id,
@start_time,
       @end_time, @translator_id, @link, @classroom_id;

WHILE @@FETCH_STATUS = 0
BEGIN
--      insert module course into corresponding tables
EXEC usp_course_module_insert @module_type_id, @module_title, @teacher_id,
@start_time,
       @end_time, @translator_id, @link,
@classroom_id;

    FETCH NEXT FROM cur_iterator INTO @module_type_id, @module_title,
@teacher_id, @start_time,
       @end_time, @translator_id, @link,
@classroom_id;
END;
--      close cursor
CLOSE cur_iterator;
DEALLOCATE cur_iterator;
END TRY
BEGIN CATCH
    IF (@@TRANCOUNT > 0)
        ROLLBACK TRAN
    THROW;
END CATCH

GO

```

'usp_course_module_insert' BT

Wstawia modul kursu do tabeli course_modules. Jesli tlumacz jest podany to wstawia tlumaczenie do tabeli slownikowej course_translations. Jesli modul jest zdalnie to dodaje link do course_online_modules, a jesli stacjonarnie to dodaje id klasy do course_classroom_modules.

```
CREATE PROCEDURE [dbo].[usp_course_module_insert](
    @course_id int,
    @module_type_id int,
    @module_title nvarchar(MAX),
    @teacher_id int,
    @start_time datetime,
    @end_time datetime,
    @translator_id int = NULL,
    @link nvarchar(MAX) = NULL,
    @classroom_id int = NULL
)
AS BEGIN
    TRY
        DECLARE @module_id int;
        SET @module_id = (SELECT ISNULL(MAX(module_id), 0) + 1 from course_modules);

        IF (@link IS NULL AND @classroom_id IS NULL) OR (@link IS NOT NULL AND
@classroom_id IS NOT NULL)
            BEGIN
                THROW 50000, 'Both "link" and "classroom_id" specified or unspecified. Class
cannot be both/neither stationary and online at the same time', 1
            end

        IF(@start_time < getdate())
            BEGIN
                THROW 50000, N'Zajęcia już się rozpoczęły, nie można ich dodać do bazy', 1
            end

        IF(dbo.fn_staff_has_free_time(@teacher_id, @start_time, @end_time) = 0 OR
        dbo.fn_staff_has_free_time(@translator_id, @start_time, @end_time) = 0)
            BEGIN
                THROW 50000, N'Nauczyciel/tłumacz jest w momencie zajęć zajęty', 1
            end

        IF( SELECT COUNT(*) FROM course_modules WHERE course_id=@course_id AND
        dbo.fn_two_datetimes_intersect(start_time, end_time,
        @start_time, @end_time) = 1) > 0
            BEGIN
                THROW 50000, N'Moduły kursu o tym samym id kolidują ze sobą', 1
            end
    end
```

```

--      ADD TO MAIN TABLE
INSERT INTO course_modules(module_id, course_id, module_type_id,
                           module_title, teacher_id, start_time, end_time)
VALUES (@module_id, @course_id, @module_type_id, @module_title, @teacher_id,
@start_time, @end_time)

--      ADD TRANSLATOR INTO course_translations TABLE (assuming only 1 translator per
course module)
IF @translator_id IS NOT NULL
BEGIN
    INSERT INTO course_translations(module_id, translator_id)
    VALUES(@module_id, @translator_id)
end

DECLARE @module_type_name nvarchar(MAX);
SET @module_type_name = (SELECT module_type_name FROM course_module_types
                        WHERE module_type_id =
@module_type_id)

-- CHECK IF module_type_id matches given attributes
IF((@module_type_name = 'Online' AND @link IS NULL) OR
   (@module_type_name = 'Stationary' AND @classroom_id IS NULL))
BEGIN
    THROW 50000, N'Podbano złą kombinację module_type_id + link/classroom_id', 1
end

IF(@module_type_name = 'Online')
BEGIN
    INSERT INTO course_online_modules(module_id, link)
    VALUES (@module_id, @link);
end
ELSE IF(@module_type_name = 'Stationary')
BEGIN
    INSERT INTO course_classroom_modules(module_id, classroom_id)
    VALUES (@module_id, @classroom_id);
end
END TRY
BEGIN CATCH
    IF(@@TRANCOUNT > 0)
        ROLLBACK TRAN
    THROW;
END CATCH
GO

```

'usp_studies_insert' BT

Wstawia całe studia do tabeli studies. Ustawia do tego cene dla członków i nie członków w tabeli studies_price_per_class. Przyjmuje JSON z informacjami o liście zajęć, dla każdego z nich tworząc rekord w studies_classes korzystając z procedury usp_studies_class_insert.

```
CREATE PROCEDURE [dbo].[usp_studies_insert](
    @name nvarchar(MAX),
    @start_date date,
    @student_limit int,
    @member_price money,
    @non_member_price money,
    @module_list_JSON as nvarchar(MAX)
) -- BT
AS
BEGIN
    TRY
        DECLARE @studies_id int;
        SET @studies_id = (SELECT ISNULL(MAX(studies_id), 0) + 1 FROM studies);

        INSERT INTO studies(studies_id, name, start_date, student_limit)
        VALUES (@studies_id, @name, @start_date, @student_limit);

--        ADD prices
        INSERT INTO studies_price_per_classes(studies_id, member_price, non_member_price)
        VALUES (@studies_id, @member_price, @non_member_price);
--        now we want to add all courses into their corresponding tables
--        first we need to unpack module list out of json
        DECLARE @title NVARCHAR(MAX);
        DECLARE @teacher_id INT;
        DECLARE @start_time DATETIME;
        DECLARE @end_time DATETIME;
        DECLARE @translator_id INT;
        DECLARE @link NVARCHAR(MAX);
        DECLARE @classroom_id INT;
        DECLARE @module_list Table (
            title nvarchar(MAX),
            teacher_id int,
            start_time datetime,
            end_time datetime,
            translator_id int,
            link nvarchar(MAX),
            classroom_id int
        );

        INSERT INTO @module_list (title, teacher_id, start_time, end_time, translator_id,
link, classroom_id)
        SELECT title, teacher_id, start_time, end_time, translator_id, link, classroom_id
```

```

FROM OPENJSON(@module_list_JSON)
WITH (
    title nvarchar(MAX),
    teacher_id int,
    start_time datetime,
    end_time datetime,
    translator_id int,
    link nvarchar(MAX),
    classroom_id int
);

--      now create cursor and iterate over modules, adding them with procedure
DECLARE cur_iterator CURSOR FOR
SELECT title, teacher_id, start_time,
       end_time, translator_id, link, classroom_id
FROM @module_list;

OPEN cur_iterator;
FETCH NEXT FROM cur_iterator INTO @title, @teacher_id, @start_time,
    @end_time, @translator_id, @link, @classroom_id;

WHILE @@FETCH_STATUS = 0
BEGIN
--      insert module course into corresponding tables
EXEC usp_studies_class_insert @studies_id, @title, @teacher_id, @start_time,
    @end_time, @translator_id, @link, @classroom_id;

    FETCH NEXT FROM cur_iterator INTO @title, @teacher_id, @start_time,
        @end_time, @translator_id, @link,
@classroom_id;
    END;

--      close cursor
CLOSE cur_iterator;
DEALLOCATE cur_iterator;
END TRY
BEGIN CATCH
    IF (@@TRANCOUNT > 0)
        ROLLBACK TRAN
    THROW;
END CATCH
GO

```

'usp_studies_class_insert' BT

Wstawia pojedyncze zajęcia na studiach do tabeli studies_classes. Jeśli translator jest podany, to wstawia go do tabeli słownikowej studies_class_translators. Jeśli link jest podany, to dodaje zajęcia do studies_online_classes, a jeśli sala jest podana, to do studies_classroom_classes.

```
CREATE PROCEDURE [dbo].[usp_studies_class_insert](
    @studies_id int,
    @title nvarchar(MAX),
    @teacher_id int,
    @start_time datetime,
    @end_time datetime,
    @translator_id int = NULL,
    @link nvarchar(MAX) = NULL,
    @classroom_id int = NULL
)
AS BEGIN
    TRY
        IF (@link IS NULL AND @classroom_id IS NULL) OR (@link IS NOT NULL AND
@classroom_id IS NOT NULL)
            BEGIN
                THROW 50000, 'Both "link" and "classroom_id" specified or unspecified. Class
cannot be both/neither stationary and online at the same time', 1
            end

        IF(@start_time < getdate())
            BEGIN
                THROW 50000, N'Zajęcia już się rozpoczęły, nie można ich dodać do bazy', 1
            end

        IF( SELECT COUNT(*) FROM studies_classes WHERE studies_id=@studies_id AND
dbo.fn_two_datetimes_intersect(start_time, end_time,
@start_time, @end_time) = 1) > 0
            BEGIN
                THROW 50000, N'Moduły studiów o tym samym id kolidują ze sobą', 1
            end

        IF(dbo.fn_staff_has_free_time(@teacher_id, @start_time, @end_time) = 0 OR
dbo.fn_staff_has_free_time(@translator_id, @start_time, @end_time) = 0)
            BEGIN
                THROW 50000, N'Nauczyciel/tłumacz jest w momencie zajęć zajęty', 1
            end

        DECLARE @class_id int;
        SET @class_id = (SELECT ISNULL(MAX(class_id), 0) + 1 FROM studies_classes);

        INSERT INTO studies_classes(class_id, studies_id, title, teacher_id, start_time,
end_time)
```

```

VALUES (@class_id, @studies_id, @title, @teacher_id, @start_time, @end_time);

--      ADD translator if its not null (if its set)
IF(@translator_id IS NOT NULL)
BEGIN
    INSERT INTO studies_class_translators(class_id, translator_id)
    VALUES (@class_id, @translator_id)
end

--      ADD TO ONLINE IF LINK SPECIFIED
IF(@link IS NOT NULL)
BEGIN
    INSERT INTO studies_online_classes(class_id, link)
    VALUES (@class_id, @link)
end

--      ADD TO STATIONARY IF CLASSROOM SPECIFIED
IF(@classroom_id IS NOT NULL)
BEGIN
    INSERT INTO studies_classroom_classes(class_id, classroom_id)
    VALUES (@classroom_id, @classroom_id)
end
END TRY
BEGIN CATCH
    IF(@@TRANCOUNT > 0)
        ROLLBACK TRAN
    THROW;
END CATCH

GO

```

'usp_change_address' BT

Zmienia adres usera na podany w adres_details. Jeśli kombinacja miasto + państwo nie występuje w countries_cities, to je tam dodaje.

```
CREATE PROCEDURE [dbo].[usp_change_address] (  
    @user_id int,  
    @street nvarchar(MAX),  
    @number int,  
    @zip nvarchar(MAX),  
    @city nvarchar(63),  
    @country nvarchar(63)  
)  
AS BEGIN  
    UPDATE dbo.adress_details  
    SET street = @street, number = @number, zip = @zip,  
        city = @city, country = @country  
    WHERE user_id = @user_id  
  
    IF NOT EXISTS (SELECT * FROM countries_cities WHERE country=@country AND city=@city)  
    BEGIN  
        INSERT INTO dbo.countries_cities(country, city)  
        VALUES (@city, @country)  
    END  
END
```

'usp_change_user_data' BT

Zmienia dane w tabeli users dla usera o podanym id.

```
CREATE PROCEDURE [dbo].[usp_change_user_data](
    @user_id int,
    @user_type_id int,
    @e_mail nvarchar(MAX),
    @hashed_passwd nvarchar(MAX),
    @first_name nvarchar(MAX),
    @last_name nvarchar(MAX)
)
AS BEGIN
    UPDATE dbo.users
    SET user_type_id=@user_type_id, e_mail=@e_mail, hashed_passwd=@hashed_passwd,
        first_name=@first_name, last_name=@last_name
    WHERE user_id = @user_id
END
```

'usp_add_presence' BT

Dodaje obecność danemu userowi na dane id usługi oraz jego typ. Jeśli is_excused = 1, to wstawia ją jako obecność która odrabia inne zajęcia w przypadku studiów.

```
CREATE PROCEDURE [dbo].[usp_add_presence](
    @service_id int,
    @service_type_name nvarchar(MAX),
    @customer_id int,
    @is_excused bit = 0
)
AS BEGIN
    IF(@service_type_name != 'course' AND @service_type_name != 'studies_classes')
    BEGIN
        THROW 50000, 'Wrong service type name', 1
    end
    IF(@service_type_name = 'course')
    BEGIN
        IF NOT EXISTS (SELECT * FROM course_modules where module_id=@service_id)
        BEGIN
            THROW 50000, 'Course module with such id does not exist', 1
        end

        INSERT INTO course_presences(module_id, customer_id)
        VALUES (@service_id, @customer_id)
    end
    IF(@service_type_name = 'studies_classes')
    BEGIN
        IF NOT EXISTS (SELECT * FROM studies_classes where class_id=@service_id)
        BEGIN
            THROW 50000, 'Studies classes with such id do not exist', 1
        end

        INSERT INTO studies_presences(class_id, customer_id, is_excused_absence)
        VALUES (@service_id, @customer_id, @is_excused)
    end
end
GO
```

Wyzwalacze

'trg_delete_details_on_order_delete' - RM

Podczas usuwania całego zamówienia, usuwa powiązane z nim rekordy.

```
CREATE TRIGGER trg_delete_details_on_order_delete
```

```
ON orders INSTEAD OF DELETE AS
```

```
BEGIN
```

```
    /* usuwaj szczegóły kolejnych zamówień */
```

```
    DECLARE cursor_order_id CURSOR
```

```
    FOR SELECT
```

```
        order_id
```

```
    FROM
```

```
        deleted;
```

```
    OPEN cursor_order_id;
```

```
    FETCH NEXT FROM cursor_order_id INTO
```

```
        @order_id;
```

```
    WHILE @@FETCH_STATUS=0
```

```
    BEGIN
```

```
        /* Usuwać szczegóły */
```

```
        DELETE FROM completed_payments WHERE order_id=@order_id;
```

```
        DELETE FROM pending_payments WHERE order_id=@order_id;
```

```
        DELETE FROM order_details WHERE order_id=@order_id;
```

```
        DELETE FROM orders WHERE order_id=@order_id;
```

```
        FETCH NEXT FROM cursor_order_id INTO
```

```
            @order_id;
```

```
    END;
```

```
    CLOSE cursor_order_id;
```

```
    DEALLOCATE cursor_order_id;
```

```
END
```

Role

'Accountant'- BT

```
CREATE ROLE Accountant;
```

```
-- Dodaj mu widoki
```

```
GRANT SELECT ON debtors TO Accountant;
```

```
GRANT SELECT ON events TO Accountant;
```

```
GRANT SELECT ON events_purchase_deadline TO Accountant;
```

```
GRANT SELECT ON user_completed_payment_details TO Accountant;
```

```
GRANT SELECT ON user_pending_payment_details TO Accountant;
```

```
-- Dodaj mu funkcje
```

```
GRANT EXECUTE ON fn_calc_average_order_value TO Accountant;
```

```
GRANT EXECUTE ON fn_calc_enrollment TO Accountant;
```

```
GRANT EXECUTE ON fn_get_event_participants_with_access_to TO Accountant;
```

```
GRANT EXECUTE ON fn_get_least_expensive_order_price TO Accountant;
```

```
GRANT EXECUTE ON fn_get_most_expensive_order_price TO Accountant;
```

```
-- Dodaj mu procedury
```

```
GRANT EXECUTE ON usp_change_address TO Accountant;
```

```
GRANT EXECUTE ON usp_change_user_data TO Accountant;
```

'principal'- BT

--- Dyrektor

CREATE ROLE principal;

--- Dyrektor ma uprawnienia administratora, co pozwala mu na całkowitą kontrolę nad dbo

GRANT ALL PRIVILEGES ON u_jmrozows TO principal;

'customer'- BT

--- Dodaj klienta

CREATE ROLE customer

-- Dodaj mu widoki

GRANT SELECT ON events TO customer;

GRANT SELECT ON basket_products TO customer; -- funkcje zrobić

GRANT SELECT ON absence_list_courses TO customer;

GRANT SELECT ON absence_list_studies_classes TO customer;

GRANT SELECT ON user_completed_payment_details TO customer; -- to i poniższe zastąpić funkcjami dla danego customer_id

GRANT SELECT ON user_pending_payment_details TO customer;

-- Dodaj mu funkcje

GRANT EXECUTE ON fn_get_bilocation_for_user TO customer;

GRANT EXECUTE ON fn_get_debt_for_user TO customer;

GRANT EXECUTE ON fn_get_presence TO customer;

GRANT EXECUTE ON fn_get_time_table_for_user TO customer;

GRANT EXECUTE ON fn_has_access TO customer;

-- Dodaj mu procedury

GRANT EXECUTE ON usp_orders_insert TO customer;

GRANT EXECUTE ON usp_change_address TO customer;

GRANT EXECUTE ON usp_change_user_data TO customer;

GRANT EXECUTE ON usp_complete_payment TO customer;

GRANT EXECUTE ON usp_delete_pending_order TO customer;

'Teacher'- BT

```
CREATE ROLE Teacher;
```

```
-- Dodaj mu widoki
```

```
GRANT SELECT ON events TO Teacher;
```

```
GRANT SELECT ON absence_list_courses TO Teacher;
```

```
GRANT SELECT ON absence_list_studies_classes TO Teacher;
```

```
-- Dodaj mu funkcje
```

```
GRANT EXECUTE ON dbo.fn_get_presence TO Teacher;
```

```
GRANT EXECUTE ON dbo.fn_get_staff_time_table_for_teacher TO Teacher;
```

```
GRANT EXECUTE ON dbo.fn_get_event_participants_with_access_to TO Teacher;
```

```
-- Dodaj mu procedury
```

```
GRANT EXECUTE ON usp_change_address To Teacher;
```

```
GRANT EXECUTE ON usp_change_user_data To Teacher;
```

```
GRANT EXECUTE ON usp_add_presence TO Teacher;
```

'Translator'- BT

```
CREATE ROLE Translator;
```

```
-- Dodaj mu widoki
```

```
GRANT SELECT ON events TO Translator;
```

```
-- Dodaj mu funkcje
```

```
GRANT EXECUTE ON fn_get_staff_time_table_for_translator TO Translator;
```

```
-- Dodaj mu procedury
```

```
GRANT EXECUTE ON usp_change_address TO Translator;
```

```
GRANT EXECUTE ON usp_change_user_data TO Translator;
```

'unsigned_customer'- BT

--- Klient niezarejestrowany

CREATE ROLE unsigned_customer;

-- Dodaj mu widoki (przeglądanie oferty)

GRANT SELECT ON events TO unsigned_customer;

-- Dodaj mu procedury (możliwość rejestracji)

GRANT EXECUTE ON usp_users_insert TO unsigned_customer

Indeksy

'Indeks service_types '- JM

Oddaje nam indeksy typów usług

```
CREATE INDEX service_types
ON service_types (service_type_id,service_type_name);
```

'Indeks studies_limitation '- JM

Indeks odpowiedzialny za ilość miejsc na danym kierunku studiów

```
CREATE INDEX studies_limitation
ON studies (studies_id,student_limit);
```

'Indeks teacher_for_studies'- JM

Indeks odpowiedzialny za nauczycieli dla danej klasy na danym kierunku studiów

```
CREATE INDEX teacher_for_studies
ON studies_classes (studies_id,class_id,teacher_id);
```

'Indeks teacher_for_webinars'- JM

Indeks odpowiedzialny za nauczycieli dla danego webinaru

```
CREATE INDEX teacher_for_webinars
ON webinars (webinar_id,teacher_id);
```

'Indeks teacher_language'- JM

Indeks odpowiedzialny za nauczycieli i przypisane im id języka

```
CREATE INDEX teacher_language
ON translators (teacher_id,language_id);
```

'Indeks studies_practice'- JM

Indeks odpowiedzialny za studia i praktyki przypisane do kierunku

```
CREATE INDEX studies_practice
ON studies_practice_modules (studies_id,practice_id);
```

'Indeks user_adress'- JM

Indeks odpowiedzialny za adres użytkownika

```
CREATE INDEX user_adress
ON adress_details (user_id,street,number,zip,city,country);
```

'Indeks user_login'- JM

Indeks odpowiedzialny za dane logowania danego użytkownika

```
CREATE INDEX user_login
```

```
ON users (user_id,e_mail,hashed_password);
```

‘Indeks user_contact’- JM

Indeks odpowiedzialny za metody kontaktu z danym użytkownikiem

```
CREATE INDEX user_contact
```

```
ON users (user_id,e_mail,phone_number);
```

‘Indeks com_inf’- JM

Indeks odpowiedzialny za informacje completeknych płatności

```
CREATE INDEX com_inf
```

```
ON completed_payments (order_id,payment_amount,payment_time);
```

‘Indeks course_list’- JM

Indeks odpowiedzialny za listę kursów

```
CREATE INDEX course_list
```

```
ON courses (course_id,title,subject_name);
```

‘Indeks webinar_list’- JM

Indeks odpowiedzialny za listę webinarów

```
CREATE INDEX webinar_list
```

```
ON webinars (webinar_id,title,subject_name);
```

‘Indeks studie_list’- JM

Indeks odpowiedzialny za listę kierunków studiów

```
CREATE INDEX studie_list
```

```
ON studies (studies_id,name);
```

‘Indeks practice_list’- JM

Indeks odpowiedzialny za listę praktyk na konkretnym kierunku

```
CREATE INDEX practice_list
```

```
ON studies_practice_modules (studies_id,practice_id,company_name);
```

‘Indeks course_module_list’- JM

Indeks odpowiedzialny za listę modułów dla danego kursu

```
CREATE INDEX course_module_list
```

```
ON course_modules (course_id,module_id,module_title);
```

‘Indeks pending_pay’- JM

Indeks odpowiedzialny za listę nie zapłaconych orderów w raz z ich kwotą i czasem do spłaty

```
CREATE INDEX pending_pay  
ON pending_payments (order_id,payment_amount,payment_due);
```

Przykłady

‘Przykładowe działanie koszyka.’ - RM

```
/* Wyczyść zamówienia powyżej 30 */
DELETE FROM pending_payments
WHERE order_id > 30;

DELETE FROM completed_payments
WHERE order_id > 30;

DELETE FROM order_details
WHERE order_id > 30;

DELETE FROM orders
WHERE order_id > 30;

GO

/* Koszyk zwraca listę produktów */
DECLARE @services_table SERVICES_TABLE_TYPE
INSERT INTO @services_table (service_id, service_type_id) VALUES
    (1, 1),
    (2, 1)
-- SELECT dbo.fn_sum_services_table(@services_table) -- 35.00

/* Ustalamy raty */
DECLARE @instalments_table INSTALMENTS_TABLE_TYPE
INSERT INTO @instalments_table (instalment, payment_due, payment_link)
VALUES (10.00, '2045-01-04 12:00:00', 'link1'),
    (25.00, '2049-01-04 12:00:00', 'link2')

/* Dodajemy oczekującą płatność */
EXEC usp_orders_insert
    @customer_id = 30,
    @discount = 0.1,
    @services_table = @services_table,
    @instalments_table = @instalments_table;

GO

/* Pokaż koszyk */
SELECT * FROM dbo.basket_products
WHERE order_id=31
ORDER BY payment_id

GO

/* Po otrzymaniu potwierdzenia dodajemy do zakończonych płatności */
EXEC usp_complete_payment
```

```
@payment_id = 31,  
@payment_time = '2040-01-04 13:55:55'
```

```
GO
```

```
/* Sprawdzamy rezultat */
```

```
SELECT      *  
FROM        orders  
WHERE       order_id > 30;
```

```
SELECT      *  
FROM        order_details  
WHERE       order_id > 30;
```

```
SELECT      *  
FROM        pending_payments  
WHERE       order_id > 30;
```

```
SELECT      *  
FROM        completed_payments  
WHERE       order_id > 30;
```

```
GO
```